European Journal of Operational Research 309 (2023) 1125–1144

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Production, Manufacturing, Transportation and Logistics

A branch-and-price-and-cut algorithm for the truck-based drone delivery routing problem with time windows

Yunqiang Yin[a], Dongwei Li[a], Dujuan Wang[b,*], Joshua Ignatius[c], T. C. E. Cheng[d], Sutong Wang[e,*]

[a] School of Management and Economics, University of Electronic Science and Technology of China, Chengdu 611731, China
[b] Business School, Sichuan University, Chengdu 610064, China
[c] Aston Business School, Aston University, Birmingham B47ET, UK
[d] Department of Logistics and Maritime Studies, The Hong Kong Polytechnic University, Kowloon, Hong Kong
[e] School of Management Science and Engineering, Dalian University of Technology, Dalian 116000, China

ARTICLE INFO

Article history:
Received 1 December 2022
Accepted 17 February 2023
Available online 21 February 2023

Keywords:
Logistics
Delivery
Vehicle routing
Column generation
Valid inequalities

ABSTRACT

Increasing e-commerce activities poses a tough challenge for logistics distribution. With the development of new technology, firms attempt to leverage drones for parcel delivery to improve delivery efficiency and reduce overall costs. We consider the truck-based drone delivery routing problem with time windows. In our setting, a set of trucks and drones (each truck is associated with a drone) collaborate to serve customers, where a drone can take off from its associated truck at a node, independently serve one or more customers within the time windows, and return to the truck at another node along the truck route. To solve the problem, we develop an enhanced branch-and-price-and-cut algorithm incorporating a bounded bidirectional labelling algorithm to solve the challenging pricing problem. To improve the algorithm, we use the subset-row inequalities to tighten the lower bound and apply enhancement strategies, which solve the pricing problem efficiency. We perform extensive numerical studies to evaluate the performance of the developed algorithm, assess the gain of the truck-based drone delivery over the truck-only delivery, and provide some managerial insights.

1. Introduction

The rapid development of e-commerce has led to an urgent need for fast and cost-effective deliveries. New technologies, such as unmanned aerial vehicles (or drones) have increasingly been deployed for parcel delivery in logistics distribution. For example, the Amazon Prime Air program began drone delivery service in 2013, which delivers packages weighing up to 5 pounds to customers in less than half an hour (Pierce, 2013). In February 2015, Hangzhou-based e-commerce provider Alibaba started drone delivery service in a partnership with Shanghai YTO Express, where drones are deployed to deliver tea to 450 customers around nearby cities in China (Bishop, 2015). In 2017, the drone company Flytrex partnered with AHA and launched Iceland's first drone delivery service in Reykjavik to shorten the delivery time and reduce cost (Gilchrist, 2017).

Compared with the traditional delivery trucks, the advantages of delivery drones include faster delivery speed, less prone to traffic congestion, lower transportation costs, and no need for manual drivers (Wang & Sheu, 2019; Wohlsen, 2014). However, with limited battery life, the drone is not suitable for long-distance delivery due to its limited carrying capacity rendering drone-only delivery limited in actual logistics distribution. Instead, truck-drone hybrid delivery, also called truck-based drone delivery in the literature, that takes advantage of both the truck and drone is increasingly popular in logistics distribution (Boysen et al., 2018; Gaba & Winkenbach, 2020; Kim & Moon, 2019; Murray & Chu, 2015). In 2014, UPS and Workhorse Group (Tamke & Buscher, 2021) developed a truck-based drone delivery system. This system uses specially-designed trucks collaborating with Workhorse "HorseFly" UAVs for parcel delivery. The truck has a dock in the roof and an opening for the operator to load packages (Fig. 1), and the Workhorse "HorseFly" UAV has a total runtime of 30 minutes with a carrying capacity of 10 pounds (Sanchez, 2017). After loading the drone with parcels, the retractable lid on the roof will open after the driver enters the destination information into the touchpad controller. The drone automatically delivers to its destination and returns to the truck when the delivery is complete. In this way, the delivery system allows drivers to complete more deliveries in less time through in-person and drone deliveries.

* Corresponding authors.
E-mail addresses: djwang@scu.edu.cn (D. Wang), wangsutong@mail.dlut.edu.cn (S. Wang).

https://doi.org/10.1016/j.ejor.2023.02.030
0377-2217/

Fig. 1. The specially-designed truck.

In the truck-based drone delivery, the truck plays the dual role of a transport facility that delivers parcels to customers and of a mobile launching platform for one or multiple drones (Boysen et al., 2018), which enables the truck and drone to collaborate. While the truck moves between different customer locations such that the driver can perform the traditional door-to-door delivery, the drone is dispatched from the truck at the same time to serve additional customers and then returns to the truck at a node along the truck route. The route along which a drone takes off from a truck, serves one or several customers, and then returns to the truck is referred to as drone route. From the OR perspective, the truck-based drone delivery operation gives rise to new and challenging decision-making problems, which have attracted much attention from OR researchers.

Most existing research on the truck-based drone delivery routing problem neglects the time preference of customers and assumes that a drone can only serve one customer per drone route. In reality, most customers can only accept parcels when it is convenient for them to be at home. In addition, allowing customers to specify time windows increases customer satisfaction even when they do not have a clear preference for any time slot. Moreover, with the rapid development of technologies, the flight endurance and carrying capacity of the latest generation of drones have been significantly enhanced, and each drone-route can serve more customers (Karak & Abdelghany, 2019; Luo et al., 2021). For example, the Ark Octocoper Drone of SF Express has a maximum load of 12 kilogram, and given that 86% of packages from Amazon are less than 2.27 kilogram (Thiago, 2019), such drones can comfortably support multiple deliveries. Based on these observations, we assume that customers impose time windows within which they require their parcel deliveries and that the drone can serve more than one customer per drone route. By modelling a pragmatic context, the problem becomes intractable, but we offer a solution in this paper.

We make three contributions to the existing literature as follows:

(i) We tackle the new and challenging truck-based drone delivery routing problem with time windows, where a set of truck-drone (each truck is associated with a drone) collaborate to serve customers, and each drone can serve one or more customers per drone route. We formulate the problem as an arc-based mixed integer linear program (MILP), which can solve instances with up to ten customers by the general-purpose solver CPLEX. To the best of our knowledge, no exact algorithm has been developed to solve large instances of the truck-based drone delivery routing problem with time windows.

(ii) We propose an exact solution approach based on the branch-and-price-and-cut (BPC) framework incorporating several enhancement strategies to solve the problem as follows. First, we introduce a hybrid algorithm incorporating an exact bounded bidirectional labelling algorithm and several heuristic strategies to solve the associated pricing problem, which is a variant of the elementary shortest path problem with time windows. Then, we apply the subset-row (SR) inequalities to tighten the lower bound obtained by column generation. Finally, we use a three-stage hierarchical branching scheme to guarantee a well-balanced search tree. The developed BPC algorithm with proper modifications can be used to solve variants of the truck-based drone delivery routing problem.

(iii) We perform extensive numerical studies to assess the efficacy of the developed algorithm, ascertain the benefits of considering the truck-based drone delivery, and analyze the impacts of the key model parameters on the solution to generate managerial insights.

We organize the rest of the paper as follows: Section 2 briefly reviews the related literature to position our study. Section 3 formally introduces the problem and presents the MILP formulation. Section 4 develops an enhanced BPC algorithm to solve the problem, and provides the details of several enhancement strategies incorporated into the algorithm. Section 5 discusses the results of the numerical studies to assess the efficacy of the developed algorithm and ascertain the benefits of considering the truck-based drone delivery. Finally, Section 6 concludes the paper and suggests future research topics.

2. Literature review

Research on drones in logistics has received increasing attention over the past few years. Increasingly more firms, including Wing, UPS, and Matternet, are using drones for parcel delivery (Hawkins, 2022; Landi, 2022; McNabb, 2021; Minemyer, 2019). Macrina et al. (2020) provided a comprehensive review of research on the drone delivery routing and truck-based drone delivery routing problems from 2015 to 2020. Our study focuses on the latter, so we only review the studies related to this research stream. Based on the number of trucks used, the truck-based drone delivery routing problem splits further into the single truck-drone combination and multiple truck-drone combinations (Tamke & Buscher, 2021).

2.1. Single truck-drone combination

Murray & Chu (2015) considered the travelling salesman problem with drones (TSP-D) to minimize the time to serve all the customers, where a drone assists a truck in performing the delivery service. The drone can make multiple trips, each time serving only one customer, starting and ending at either the depot or the truck. They developed a truck-first-drone-second-based heuristic to solve instances with only ten customers. Subsequent studies further investigated the TSP-D under similar assumptions. Agatz et al. (2018) proposed several fast route-first, cluster-second heuristics to solve the TSP-D, and derived the worst-case approximation ratios for the heuristics. Yurek & Ozmutlu (2018) put forward an iterative algorithm based on a decomposition approach to solve the TSP-D, where the first stage determines the truck route and the assignment of customers to the drone, and the second stage optimally determines the drone routes, and test the developed algorithm using instances with 20 customers. To minimize the total operating cost composed of the travel and mutual waiting costs, Ha et al. (2018) developed two heuristics, where the first one is adapted from the method proposed by Murray & Chu (2015) and the second one, called greedy randomized adaptive search procedure, is based on a strategy that transforms any TSP route into TSP-D solutions. Considering the effect of the parcel weight on the drone's energy

consumption, Jeong et al. (2019) developed a two-phase constructive and search heuristic algorithm, and test the performance of their algorithm against the genetic algorithm, particle swarm optimization, and simulated annealing. Wang et al. (2020) investigated the TSP-D with the objective of minimizing both total operating cost and delivery completion time, and developed an improved non-dominated sorting genetic algorithm to find the Pareto front. To solve the TSP-D to optimality, Roberti & Ruthmair (2021) developed a branch-and-price algorithm that combines *ng*-route relaxation and a three-level hierarchical branching strategy, which can solve instances with up to 39 customers.

Unlike Boysen et al. (2018); Murray & Chu (2015) focused on finding the optimal drone route given a truck route, and distinguished between trucks carrying one or more drones and whether take-off and landing nodes must overlap. They presented an efficient MILP and analysed the computational complexity of each resulting case. The TSP-D with a drone station was considered by Kim & Moon (2019), where the drones are stored in the drone station and can only take off from and return to the drone station, and the truck supplements the capacity of the drones at the station. As such, much less synchronization between the truck and drones is needed. Hence, the problem decomposes into independent travelling salesman and identical parallel-machine scheduling problems. Kang & Lee (2021) addressed another variant of the TSP-D, where the truck carries multiple drones, and the drones dispatched simultaneously from the truck at a node along the truck route to serve customers, and the truck must wait until all the drones return. They put forward an exact algorithm based on logic-based Benders decomposition, which performs better than CPLEX. There are few studies that allow multiple customers to be visited per drone route. Karak & Abdelghany (2019) investigated the multi-visit TSP-D for pick-up and delivery services, where the drones can only be launched and retrieved at drone stations, and developed a heuristic based on the classic Clarke and Wright algorithm. Luo et al. (2021) investigated the multi-visit TSP-D, where the truck is equipped with a homogeneous fleet of drones, and developed a tabu search algorithm.

2.2. Multiple truck-drone combinations

The problem corresponding to a set of truck-drone combinations, also known as the truck-based drone delivery routing problem (TD-DRP), involves the collaboration of multiple trucks and drones to serve a given set of customers is a variant of the TSP-D. Di Puglia Pugliese & Guerriero (2017) studied the TD-DRP, where each truck carries a limited number of drones, and developed a MILP that can solve instances with ten customers and two trucks. To minimize the maximum completion time of the vehicles or minimize the total completion time of all the services, Tamke & Buscher (2021) considered the TD-DRP, where each truck is associated with a drone and the drone can only take off from its associated truck, serves one customer, and returns to the truck, and put forward a branch-and-cut algorithm, which can solve instances with 20 customers when the drone flight endurance is unlimited and instances with 30 customers when the drone flight endurance is limited. Kitjacharoenchai et al. (2020) investigated the TD-DRP, where multiple drones can take off from a truck, serve one or multiple customers, and return to the same truck to minimize the total arrival time at the depot of all trucks and drones. They developed two efficient heuristics, namely drone truck route construction and large neighbourhood search to solve the problem. In addition, Kitjacharoenchai et al. (2019) studied the case where the drones can switch between different trucks, and presented an insertion-based heuristic to solve large-sized instances with 100 customers.

Analogous to Kim & Moon (2019); Kloster et al. (2023) considered the TD-DRP with multiple drone stations to minimize the makespan, where the trucks can supply some drone stations, which can then launch and operate drones to serve customers, and presented three matheuristics to solve the problem. Slightly different from Kim & Moon (2019) and Kloster et al. (2023); Wang & Sheu (2019) investigated the TD-DRP, where drones can be launched and recovered at different stations, and developed a branch-and-price algorithm that requires more than four hours to solve instances with 15 customers and two drone stations. Dayarian et al. (2020) studied a variant of the TD-DRP, where the drones re-supply the trucks whenever a delivery truck is stationary. Kuo et al. (2022) considered the TD-DRP with time windows, where drones can only serve one customer per time, and presented a variable neighborhood search algorithm that performs better than the adaptive large neighborhood search algorithm proposed by Sacramento et al. (2019).

The detailed comparison of the main problem characteristics and solution approaches of the works in this area are summarized in Table 1. To summarize, most existing studies do not consider the time windows imposed by customers, and none of the existing algorithms can fully deal with our problem. The work of Tamke & Buscher (2021) is the closest to the problem under our study but differs in two major aspects. From the modelling perspective, our major differences are we impose limits on the carrying capacity of each truck and introduce time windows, and a drone can serve more than one customer instead of one per drone route. From the solution algorithm perspective, we propose a BPC algorithm that is more efficient than the branch-and-cut algorithm.

3. Problem description and model formulation

In this section, we formally describe the problem under study, and present a arc-based mixed integer program for the problem. The main notation used in the study is listed in Table 2.

3.1. Problem description

We model the truck-based drone delivery routing problem with time windows (TD-DRPTW) as a complete directed graph $G = (N, A)$, where N is the node set (containing the origin depot 0 and end depot $n+1$, both of which denote the distribution centre and a set of customer nodes C), and A is the arc set. Associated with each customer i is a product demand d_i, and a time window $[e_i, l_i]$ that specifies the time interval within which customer i is available to receive the delivery service in the planning horizon. The starting service time at customer i must be at or later than time e_i up to l_i.

The distribution of the products from the distribution centre (depot) to customers is performed collaboratively by a fleet of homogeneous trucks and drones available at the depot, where each truck is associated with one drone exclusively. Thus, it is forbidden for a drone to take off from and land on any other truck other than its associated truck (Tamke & Buscher, 2021). Let K be the set of truck-drone combinations. Each drone has a battery with a maximum battery life of L^d. The trucks and drones differ in terms of the carrying capability, speed, and route they take between any two nodes. We denote the maximum carrying capabilities of a truck and a drone by Q^t and Q^d, respectively, with $Q^d < Q^t$. When a drone takes off from its associated truck to serve customers, it carries up to Q^d units of the products and has a fully charged battery. Given that the landing of a drone needs to fulfill certain conditions and that the demand of a customer may exceed the maximum carrying capability of a drone, some customers cannot be served by the drones (Agatz et al., 2018). We let D be the subset of customers that the drones can serve.

Table 1
Overview of vehicle routing problems with drones in recent academic literature.

Reference	#T	#D	Time window	Multi-visit	Drone capacity	Station	Objective function	Solution approach
Murray & Chu (2015)	1	1				N	completion time	heuristic
Agatz et al. (2018)	1	1				N	completion time	heuristic
Yurek & Ozmutlu (2018)	1	1				N	completion time	heuristic
Ha et al. (2018)	1	1				N	operational costs	heuristic
Jeong et al. (2019)	1	1				N	completion time	heuristic
Wang et al. (2020)	1	1				N	operational costs & completion time	heuristic
Roberti & Ruthmair (2021)	1	1				N	completion time	BP
Boysen et al. (2018)	1	n				N	completion time	Gurobi
Kim & Moon (2019)	1	n				Y	completion time	heuristic
Kang & Lee (2021)	1	n				N	completion time	BD
Karak & Abdelghany (2019)	1	n		✓		N	operational costs	heuristic
Luo et al. (2021)	1	n		✓		N	completion time	heuristic
Di Puglia Pugliese & Guerriero (2017)	m	n				N	operational costs	heuristic
Tamke & Buscher (2021)	m	m				N	completion time	BC
Kitjacharoenchai et al. (2019)	m	m				N	completion time	heuristic
Kitjacharoenchai et al. (2020)	m	n			✓	N	completion time	heuristic
Wang & Sheu (2019)	m	n		✓	✓	Y	operational costs	BP
Dayarian et al. (2020)	m	n			✓	N	number of orders	heuristic
Kloster et al. (2023)	m	n				Y	completion time	heuristic
Kuo et al. (2022)	m	m	✓			N	operational costs	heuristic
Sacramento et al. (2019)	m	m				N	operational costs	heuristic
This paper	**m**	**m**	✓	✓	✓	N	**operational costs**	**BPC**

Note: "#T" and "#D" represent the number of trucks and drones; "Time window" denotes whether the time windows of customers are considered; "Drone capacity" indicates whether the maximum load of drones is considered; "Station" indicates whether the truck and drone interact with each other with the help of Station, "N" means no, "Y" means yes; and "Gurobi" means solving directly using commercial solver Gurobi, "BD" means Benders decomposition algorithm, "BC" means branch-and-cut algorithm, "BP" means branch-and-price algorithm, "BPC" means branch-and-price-and-cut algorithm.

Table 2
Sets, parameters and decision variables.

Sets	
$K = \{1, \ldots, \bar{k}\}$	The set of truck-drone combinations
$C = \{1, \ldots, n\}$	The set of customer nodes
$N = \{0, n+1\} \cup C$	The set of nodes
$N^+ = \{0, 1, \ldots, n\}$	The set of customer and origin depot nodes
$N^- = \{1, \ldots, n, n+1\}$	The set of customer and end depot nodes
$\delta^+(i)$	The set of successor nodes of node i
$\delta^-(i)$	The set of predecessor nodes of node i
$A = \{(i,j) \mid i \in N^+, j \in N^-\}$	The set of arcs
D	The set of customers that can be served by the drones
Parameters	
t^t_{ij}	The travel time between the locations of customers i and j for a truck
t^d_{ij}	The travel time between the locations of customers i and j for a drone
F	The fixed cost of dispatching a truck-drone combination to serve customers
c^t	The travel cost per unit travel time for the trucks
c^d	The travel cost per unit travel time for the drones
Q^t	The maximum carrying capability of a truck
Q^d	The maximum carrying capability of a drone
L^t	The maximum total duration of the truck route
L^d	The maximum battery life
e_i	The earliest start service time at customer i
l_i	The latest start service time at customer i
d_i	The product demand of customer i
Decision variables	
x_{ijk}	A binary variable equal to 1 if truck $k \in K$ traverses arc $(i,j) \in A$ alone; 0 otherwise
y_{ijk}	A binary variable equal to 1 if drone $k \in K$ traverses arc $(i,j) \in A$ alone; 0 otherwise
z_{ijk}	A binary variable equal to 1 if truck $k \in K$ traverses arc $(i,j) \in A$ together with the drone; 0 otherwise
w_{ik}	The cumulative load of truck $k \in K$ or cumulative load of drone $k \in K$ after its latest take-off upon serving node $i \in N$
u_{ik}	The cumulative flight time when drone $k \in K$ arrives at node $i \in N$ after its latest take-off
τ^T_{ik}	The earliest arrival time of truck $k \in K$ at node $i \in N$
τ^D_{ik}	The earliest arrival time of drone $k \in K$ at node $i \in N$
s^T_{ik}	A binary variable equal to 1 if truck $k \in K$ visits node $i \in N$ alone; 0 otherwise
s^D_{ik}	A binary variable equal to 1 if drone $k \in K$ visits node $i \in N$ alone; 0 otherwise
s^C_{ik}	A binary variable equal to 1 if node $i \in N$ is a separation node or a rendezvous node in a synthetic-route of the truck-drone combination $k \in K$; 0 otherwise
s^E_{ik}	A binary variable equal to 1 if node $i \in N$ is visited by truck $k \in K$ together with the drone, which means that the truck enters and leaves node $i \in N$ together with the drone; 0 otherwise

Following (Dayarian et al., 2020), we assume that a drone can fly in a straight line from one location to another, while a truck route is confined by the road network, and that the speed of a drone is faster than that of a truck. Specifically, let v^t and v^d be the speeds of a truck and a drone so that $v^t = \alpha v^d$ with $\alpha \le 1$. For each arc $(i, j) \in A$, let d_{ij}^t and d_{ij}^d be the travel distances between the locations of customers i and j for a truck and a drone, respectively, such that $d_{ij}^t = \beta d_{ij}^d$ with $\beta \ge 1$. Therefore, the travel times between the locations of customers i and j for a truck and a drone are $t_{ij}^t = \frac{d_{ij}^t}{v^t}$ and $t_{ij}^d = \frac{d_{ij}^d}{v^d} = \frac{\alpha d_{ij}^t}{\beta v^t} = \frac{\alpha t_{ij}^t}{\beta}$, respectively. Here, both the travel times t_{ij}^t and t_{ij}^d include the service time at node j. The service time at a node usually consists of the unloading time for a truck, and the landing time, unloading time, and taking-off time for a drone. Following Tamke & Buscher (2021), we assume that drone loading and battery swapping are completely autonomous, and we consider the loading time, battery swapping time, and take-off time to be negligible for a drone when it takes off from its associated truck to serve customers because all these activities can be simultaneously performed when the truck serves a customer. Without loss of generality, we assume that both the travel distances of the truck and drone satisfy the triangle inequality, implying that the triangle inequality holds for the travel times of the trucks and drones.

Each customer (node) is served (visited) exactly once, either by a truck or a drone. When a node is visited by a truck alone, it is referred to as a *truck node*, while when a node is visited by a drone alone, it is referred to as a *drone node*. When a node is visited by a truck-drone combination, it is referred to as a *combined node*. The node at which the truck-drone combination separates is referred to as a *separation node*, and the node at which the truck-drone combination rejoins is referred to as a *rendezvous node*. The truck and drone in combination are allowed to wait for each other whenever one arrives at a rendezvous node before the other. Similar to Tamke & Buscher (2021), we assume that the drone can freely wait for the truck on the ground.

A synthetic route of a truck-drone combination is the route of a truck that departs from the depot, serves some customers exactly once, and finally returns to the depot, together with a series of routes of drones, each of which departs from a separation node, serves some customers exactly once, and then returns to a rendezvous node. If the time window and capacity limit are met and the total duration of a truck does not exceed the threshold L^t due to the working time regulations, the truck route is feasible. If the time window and capacity limit are met and the flight time of a drone does not exceed L^d due to the limitation of battery life, the drone route is feasible.

Dispatching a truck-drone combination to serve customers incurs a fixed cost F. We denote the travel costs per unit travel time for the trucks and drones by c^t and c^d, respectively. Thus, the objective of the TD-DRPTW is to find the optimal synthetic routes that minimize the total operating cost, comprising the fixed cost of dispatching the truck-drone combinations, and the travel costs of the trucks and drones.

Figure 2 depicts a feasible synthetic-route, which covers eight nodes, where the truck visits nodes 1, 4, 5, 6, 8 and 9, and the drone visits nodes 2, 3, and 7.

3.2. Arc-based mixed integer linear program

Using the notation listed in Table 2, we model the TD-DRPTW as an arc-based MILP, denoted as AB-MILP, as follows:

$$\min \sum_{(i,j)\in A}\sum_{k\in K}\left[c^t t_{ij}^t (x_{ijk}+z_{ijk}) + c^d t_{ij}^d y_{ijk}\right] + F\sum_{j\in\delta^+(0)}\sum_{k\in K}(x_{0jk}+z_{0jk}) \tag{1}$$

$$\text{s.t.} \sum_{j\in\delta^+(0)}(x_{0jk}+z_{0jk}) = \sum_{i\in\delta^-(n+1)}(x_{i,n+1,k}+z_{i,n+1,k}) \le 1, \forall k\in K \tag{2}$$

$$\sum_{j\in\delta^+(0)}(y_{0jk}+z_{0jk}) = \sum_{i\in\delta^-(n+1)}\left(y_{i,n+1,k}+z_{i,n+1,k}\right) \le 1, \forall k\in K \tag{3}$$

$$\sum_{j\in\delta^+(0)} x_{0jk} = \sum_{j\in\delta^+(0)} y_{0jk}, \forall k\in K \tag{4}$$

$$\sum_{j\in\delta^-(n+1)} x_{j,n+1,k} = \sum_{j\in\delta^-(n+1)} y_{j,n+1,k}, \forall k\in K \tag{5}$$

$$w_{ik} \le Q^t, \forall i\in N, k\in K \tag{6}$$

$$w_{jk} + (s_{jk}^D - 1)Q^t \le Q^d, \forall j\in N, k\in K \tag{7}$$

$$w_{jk} \ge w_{ik} + \left(y_{ijk} + s_{ik}^D + s_{jk}^D - 3\right)Q^t + d_j, \forall i\in N^+, j\in N^-, k\in K \tag{8}$$

$$w_{jk} \ge \left(y_{ijk} + s_{ik}^C + s_{jk}^D - 3\right)Q^t + d_j, \forall i\in N^+, j\in N^-, k\in K \tag{9}$$

$$w_{jk} \ge w_{ik} + (x_{ijk} + z_{ijk} - 1)Q^t + d_j, \forall i\in N^+, j\in N^-, k\in K \tag{10}$$

$$w_{jk} \ge w_{ik} + w_{lk} + (x_{ijk} + y_{ljk} - 2)Q^t + d_j, \forall i\in N^+, j\in N^-, l\in D, k\in K \tag{11}$$

$$u_{jk} \ge u_{ik} + t_{ij}^d + (y_{ijk} + s_{ik}^D - 2)L^d, \forall i\in N^+, j\in N^-, k\in K \tag{12}$$

$$u_{jk} \ge t_{ij}^d + (y_{ijk} + s_{ik}^C - 2)L^d, \forall i\in N^+, j\in D, k\in K \tag{13}$$

$$u_{jk} \le L^d, \forall j\in N, k\in K \tag{14}$$

$$\tau_{jk}^T \ge \tau_{ik}^T + t_{ij}^t + (x_{ijk} + z_{ijk} - 1)L^t, \forall i\in N^+, j\in N^-, k\in K \tag{15}$$

$$\tau_{jk}^T \ge e_i + t_{ij}^t + (x_{ijk} + z_{ijk} - 1)L^t, \forall i\in N^+, j\in N^-, k\in K \tag{16}$$

$$\tau_{jk}^D \ge \tau_{ik}^D + t_{ij}^d + (s_{ik}^D + y_{ijk} - 2)L^t, \forall i\in N^+, j\in N^-, k\in K \tag{17}$$

$$\tau_{jk}^D \ge e_i + t_{ij}^d + (s_{ik}^D + y_{ijk} - 2)L^t, \forall i\in N^+, j\in N^-, k\in K \tag{18}$$

$$\tau_{jk}^D \ge \tau_{ik}^T + t_{ij}^d + (s_{ik}^C + y_{ijk} - 2)L^t, \forall i\in N^+, j\in D, k\in K \tag{19}$$

$$\tau_{jk}^T \ge \tau_{ik}^D + t_{ij}^t + (s_{ik}^C + x_{ijk} + z_{ijk} - 2)L^t, \forall i\in N^+, j\in N^-, k\in K \tag{20}$$

$$\tau_{jk}^T \le l_j \sum_{i\in\delta^-(j)}(x_{ijk}+z_{ijk}), \forall j\in N, k\in K \tag{21}$$

$$\tau_{jk}^D \le l_j s_{jk}^D, \forall j\in N, k\in K \tag{22}$$

$$\tau_{n+1,k}^T \le L^t, \forall k\in K \tag{23}$$

$$\sum_{i\in\delta^-(j)} x_{ijk} \le s_{jk}^T + s_{jk}^C, \forall j\in N, k\in K \tag{24}$$

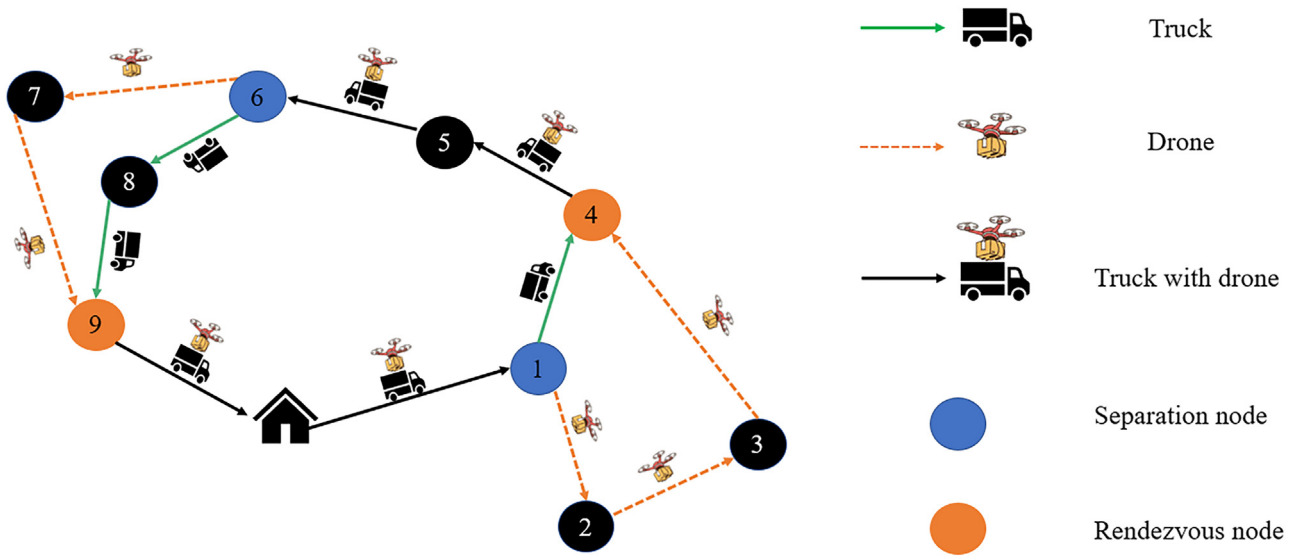

Fig. 2. A feasible synthetic-route of TD-DRPTW.

$$\sum_{l\in\delta^+(j)} x_{jlk} \le s^T_{jk} + s^C_{jk}, \forall j \in N, k \in K \tag{25}$$

$$\sum_{i\in\delta^-(j)} y_{ijk} \le s^D_{jk} + s^C_{jk}, \forall j \in N, k \in K \tag{26}$$

$$\sum_{l\in\delta^+(j)} y_{jlk} \le s^D_{jk} + s^C_{jk}, \forall j \in N, k \in K \tag{27}$$

$$\sum_{i\in\delta^-(j)} z_{ijk} \le s^E_{jk} + s^C_{jk}, \forall j \in N, k \in K \tag{28}$$

$$\sum_{l\in\delta^+(j)} z_{jlk} \le s^E_{jk} + s^C_{jk}, \forall j \in N, k \in K \tag{29}$$

$$\sum_{i\in\delta^-(j)} z_{ijk} + \sum_{l\in\delta^+(j)} z_{jlk} - 1 \le s^E_{jk}, \forall j \in N, k \in K \tag{30}$$

$$\sum_{k\in K} \left(s^E_{jk} + s^C_{jk} + s^T_{jk} + s^D_{jk}\right) = 1, \forall j \in C \tag{31}$$

$$\begin{aligned}\sum_{i\in\delta^-(j)} \left(x_{ijk} + y_{ijk} + 2z_{ijk}\right) &= 2s^E_{jk} + 2s^C_{jk} + s^T_{jk} + s^D_{jk} \\ &= \sum_{l\in\delta^+(j)} \left(x_{jlk} + y_{jlk} + 2z_{jlk}\right), \\ &\forall j \in C, k \in K\end{aligned} \tag{32}$$

$$x_{ijk} + y_{ijk} + z_{ijk} \le 1, \forall (i,j) \in A, k \in K \tag{33}$$

$$\sum_{i\in\delta^-(j)} z_{ijk} \le 1 - \sum_{i\in\delta^-(j)} y_{ijk}, \forall j \in N, k \in K \tag{34}$$

$$x_{ijk}, y_{ijk}, z_{ijk}, s^D_{ik}, s^T_{ik}, s^C_{ik}, s^E_{ik} \in \{0,1\}, \forall i \in N, k \in K \tag{35}$$

$$\tau^T_{ik}, \tau^D_{ik}, w^T_{ik}, w^D_{ik} \ge 0, \forall i \in N, k \in K \tag{36}$$

The objective function (1) minimizes the total operating cost. Constraints (2) and (3) ensure that each used truck-drone combination must depart from the origin depot and return to the end depot, whereas constraints (4) and (5) guarantee that each used truck-drone combination either departs from the origin depot together or separately, and either returns to the end depot together or separately, respectively.

Constraints (6)–(11) together regulate the load resource. Constraints (6) and (7) state that the cumulative load of customer demand cannot exceed the maximum carrying capacity of the truck and the cumulative load of customer demand of a drone after the latest take-off cannot exceed the maximum carrying capacity of the drone. Constraints (8) and (9) define the cumulative load of a drone after its latest take-off, and constraints (10) and (11) give the cumulative load of customer demand along a partial synthetic-route.

Constraints (12)–(14) together regulate the power resource. Constraints (12) and (13) define the cumulative flight time of a drone after its latest take-off, and constraint (14) ensures that the cumulative flight time of a drone cannot exceed the limit of the battery life.

Constraints (15)–(23) together regulate the time resource. Constraints (15)–(20) define the earliest arrival times of a truck and a drone at the nodes they respectively serve. Constraints (21) and (22) state that the time windows cannot be violated. Constraint (23) ensures that the total duration of the truck route does not exceed the threshold L^t.

Constraints (24)–(30) give the logical relationships among the binary variables x_{ijk}, y_{ijk}, z_{ijk} and the binary variables s^D_{ik}, s^T_{ik}, s^C_{ik}, s^E_{ik}. Constraints (24) and (25) ensure that if a node has arc inflows (resp., outflows) traversed by a truck, the node is a separation/rendezvous (resp., truck) node. Constraints (26) and (27) ensure that if a node has arc inflows (resp., outflows) traversed by a drone, the node is a separation/rendezvous (resp., drone) node. Constraints (28) and (29) ensure that if a node has arc inflows (resp., outflows) traversed by a truck together with the drone, the node is a separation/rendezvous (resp., combined but not a separation) node. Constraints (30) define the variables s^E_{ik} by variables z_{ijk}.

Constraints (31) ensure that each node is visited exactly once by a truck-drone combination. Constraints (32) give the flow balances at different kinds of nodes. Constraints (33) guarantee that each arc is visited at most once. Constraints (34) state that a node cannot be simultaneously visited by a drone alone and the drone together

with its truck. Constraints (35) and (36) define the feasible ranges of the values for the decision variables.

4. Solution approach

We develop an enhanced BPC algorithm to solve the TD-DRPTW. In the algorithm, we compute the lower bounds in the branch-and-bound search tree by column generation based on a set partitioning formulation of the problem. We dynamically add the subset-row (SR) inequalities to tighten the bounds. We discuss the main components of the BPC algorithm in the following.

4.1. Set partitioning formulation

We develop a set partitioning formulation of the problem. Let R_s be the set of all the feasible synthetic-routes of the truck-drone combinations. Denote by c_r the total operating cost of synthetic-route $r \in R_s$, and by a_{ir} a binary parameter equal to 1 if and only if node $i \in N$ is covered by synthetic-route $r \in R_s$. For any $r \in R_s$, let λ_r be a binary variable equal to 1 if and only if synthetic-route r is used in the optimal solution.

Based on the above notation, we formulate the set partitioning problem, which we call the *master problem* (MP), as follows:

$$\min \sum_{r \in R_s} c_r \lambda_r \tag{37}$$

$$\text{s.t.} \sum_{r \in R_s} a_{ir} \lambda_r = 1, \forall i \in C, \tag{38}$$

$$\sum_{r \in R_s} \lambda_r \leq \bar{k}, \tag{39}$$

$$\lambda_r \in \{0, 1\}, \forall r \in R_s. \tag{40}$$

To tighten the lower bound on the linear relaxation of the MP, denoted as LMP, we consider the SR inequalities introduced by Jepsen et al. (2008). An SR inequality for our problem is defined by a node set $S \subseteq C$ and an integer p such that $1 < p \leq |S|$, which can be written as

$$\sum_{r \in R_s} \left\lfloor \frac{1}{p} \sum_{i \in S} a_{ir} \right\rfloor \lambda_r \leq \left\lfloor \frac{|S|}{p} \right\rfloor$$

In particular, we restrict ourselves to those SR inequalities by complete enumeration for $|S| = 3$ and $p = 2$. In this case, the SR inequality reduces to

$$\sum_{r \in R_s} \left\lfloor \frac{1}{2} \sum_{i \in S} a_{ir} \right\rfloor \lambda_r \leq 1 \tag{41}$$

which indicates that at most one synthetic-route visiting more than one node in S can be selected.

4.2. Column and cut generation

To efficiently solve the LMP involving a large number of variables, we apply the column-and-cut generation (CCG) algorithm. Initially, only a subset of feasible synthetic-routes and SR inequalities are included in the model, which we call the *restricted main problem*, denoted as RLMP. Solving the RLMP yields primitive and dual solutions. The pricing problem constructed from the dual solution attempts to identify columns (variables) whose reduced costs are negative among the columns that have not been included in the RLMP or prove that such columns do not exist. In the former case, we append the columns with the most negative reduced cost to the current RLMP, triggering a new iteration. In the latter case, if the primal solution is fractional, we try to identify some valid SR inequalities to separate the primal solution from the domain of the LMP. If valid SR inequalities are found, we append them to the current RLMP, triggering a new iteration. Otherwise, the algorithm stops.

Let Ω be the set of the identified SR inequalities, and let $\pi_i, i \in C$, σ, and $\zeta_S, S \in \Omega$, be the dual variables associated with constraints (38), (39), and (41), respectively. Then the reduced cost of the variable $\lambda_r, r \in R_s$, is

$$\begin{aligned} \bar{c}_r &= c_r - \sum_{i \in C} a_{ir} \pi_i - \sum_{S \in \Omega} \left\lfloor \frac{1}{2} \sum_{i \in S} a_{ir} \right\rfloor \zeta_S - \sigma \\ &= \sum_{(i,j) \in A} \left[\bar{c}^t_{ij} (x_{ij} + z_{ij}) + \bar{c}^d_{ij} y_{ij} \right] - \sum_{S \in \Omega} \left\lfloor \frac{1}{2} \sum_{i \in S} a_{ir} \right\rfloor \zeta_S + F - \sigma \end{aligned} \tag{42}$$

where $\bar{c}^t_{ij} = c^t t^t_{ij} - \pi_j$, $\bar{c}^d_{ij} = c^d t^d_{ij} - \pi_j$, x_{ij}, y_{ij}, and z_{ij} are binary variables as defined in the AB-MILP by removing the index k.

Thus, the pricing problem can be defined as finding a feasible synthetic-route of a truck-drone combination with the minimum reduced cost, which is an extension of the strongly *NP*-hard classical shortest path problem with time windows.

4.2.1. Bounded bidirectional labelling algorithm

In this section we adapt the labelling algorithm proposed by Righini & Salani (2006) to solve the pricing problem. The bidirectional search strategy is a commonly used technique to speed up the label setting algorithm. We perform a bidirectional search such that the labels can be propagated both forward from the origin depot 0 to its successors until the earliest arrival time is greater than $\frac{L^t}{2}$ and backward from the end depot $n+1$ to its predecessors until the latest departure time is less than $\frac{L^t}{2}$. At the end of the algorithm, the forward and backward labels are joined to generate complete synthetic-routes.

Let $\pi_0 = \pi_{n+1} = \frac{F-\sigma}{2}$, n_S be the number of SR inequalities with nonzero dual values in the dual solution to the current RLMP, and denote by SR_l the lth SR inequality with a nonzero dual value and by $S(l)$ the customer subset corresponding to the SR inequality SR_l. In the following we present the forward and backward labelling algorithms in detail.

The forward labelling algorithm

Forward label definition. For each pair of $(i_t, i_d) \in N \times N$, let $Lf_{i_t i_d}$ be the set of forward labels, each of which encodes a partial synthetic-route that extends forwardly from the origin depot 0 to its successors and ends at nodes i_t and i_d, where i_t and i_d are the last nodes to visit by the truck and drone, respectively. Each label $lf_r = (tf^t, tf^d, wf^t, wf^d, vf^d, uf, (sf^{SR_l})_{l=1,\ldots,n_S}, cf) \in Lf_{i_t i_d}$ is composed of the following semantics:

- tf^t: the earliest arrival time of the truck at node i_t
- tf^d: the earliest arrival time of the drone at node i_d
- wf^t: the residual load of the truck after serving node i_t (the drone is unloaded when it is carried on the truck and fully loaded when it takes off from the truck, so the residual load of the truck is the sum of the residual load of the drone and the residual load of the truck at rendezvous nodes, and is the difference between the residual load of the truck and the maximum carrying capacity of the drone at separation nodes)
- wf^d: the residual load of the drone after serving node i_d
- vf^d: the residual flight time of the drone after serving node i_d (the residual flight time is 0 at rendezvous nodes, and is L^d when it takes off from the truck)
- uf: the subset of nodes, each of which is either visited in the partial synthetic-route r or unreachable from r due to the delivery capacity or time window constraint

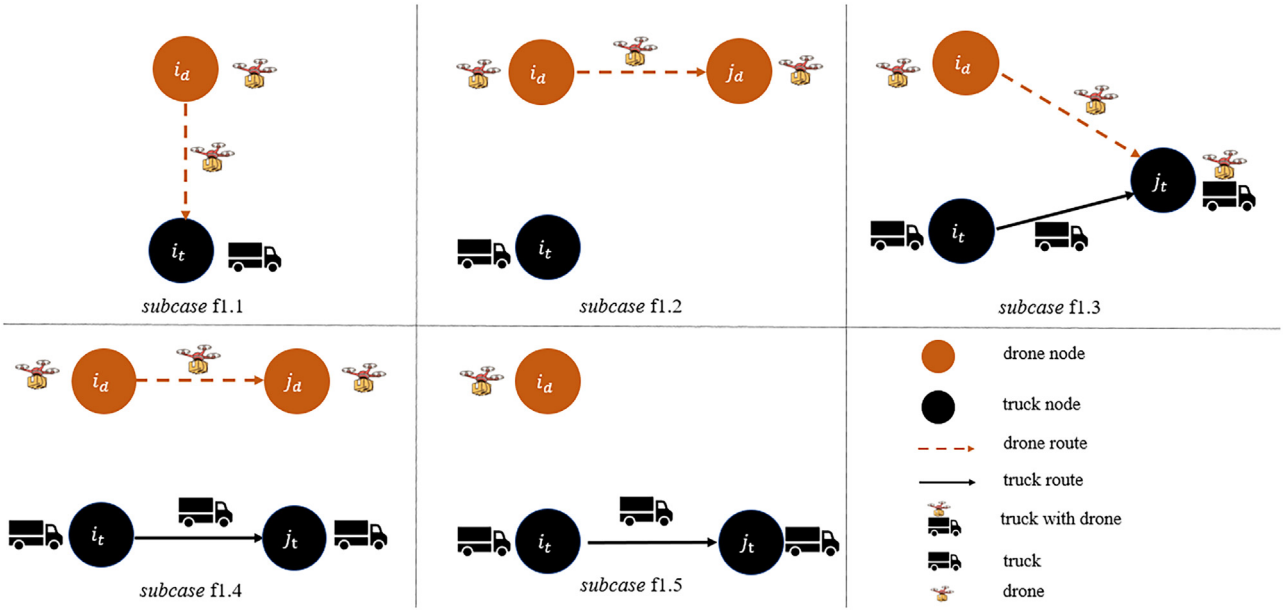

Fig. 3. The subcases of Case 1 in forward label extension.

sf^{SR_l}: the state of SR inequality SR_l along the partial synthetic-route r

cf: the accumulated reduced cost along the partial synthetic-route r

We say that a node j is unreachable from the partial synthetic-route r if one of the following conditions is satisfied:

(i) $wf^t + wf^d < d_j$;
(ii) $\max\{tf^t, e_{i_t}\} + t^t_{i_t j} > l_j$ if $j \in N^- \setminus D$; and
(iii) $\max\{tf^t, e_{i_t}\} + t^t_{i_t j} > l_j$ and $\max\{tf^d, e_{i_d}\} + t^d_{i_d j} > l_j$ if $j \in D$.

Forward label extension

The label set Lf_{00} is initialized with $(0, 0, Q^t, 0, 0, \emptyset, \mathbf{0}, 0)$. For each label $lf_r = (tf^t, tf^d, wf^t, wf^d, vf^d, uf, (sf^{SR_l})_{l=1,\dots,n_S}, cf) \in Lf_{i_t i_d}$, to generate new states, there are two cases depending on whether the drone is separated from the truck to consider.

Case f1: $i_t \neq i_d$, implying that the drone is separated from the truck. In this case, we further consider five subcases (see Fig. 3 for detail).

Subcase f1.1: The truck and drone rejoin at node i_t. In this subcase, a new label $lf_{r'} = (tf'^t, tf'^d, wf'^t, wf'^d, vf'^d, uf', (sf'^{SR_l})_{l=1,\dots,n_S}, cf')$ is generated, where

$$tf'^t = tf^t \tag{f1.1}$$

$$tf'^d = \max\{tf^d, e_{i_d}\} + t^d_{i_d i_t} \tag{f1.2}$$

$$wf'^t = wf^t + wf^d \tag{f1.3}$$

$$wf'^d = 0 \tag{f1.4}$$

$$vf'^d = 0 \tag{f1.5}$$

$$uf' = uf \cup Uf_{r'} \tag{f1.6}$$

$$sf'^{SR_l} = sf^{SR_l}, l = 1, \dots, n_S \tag{f1.7}$$

$$cf' = cf + c^d t^d_{i_t i_d} - \frac{\pi_{i_d}}{2} \tag{f1.8}$$

Here $Uf_{r'}$ is the subset of the nodes in $N^- \setminus uf$ that are unreachable from the partial synthetic-route r'. If $t^d_{i_t i_d} \le vf^d$, label $lf_{r'}$ is added to the label set $Lf_{i_t i_t}$.

Subcase f1.2: The drone continues to serve another customer, and the truck stays at node i_t. In this subcase, for each node $j_d \in (D \cup \{n+1\}) \setminus uf$, a new label $lf_{r'} = (tf'^t, tf'^d, wf'^t, wf'^d, vf'^d, uf', (sf'^{SR_l})_{l=1,\dots,n_S}, cf')$ is generated, where

$$tf'^t = tf^t \tag{f1.9}$$

$$tf'^d = \max\{tf^d, e_{i_d}\} + t^d_{i_d j_d} \tag{f1.10}$$

$$wf'^t = wf^t \tag{f1.11}$$

$$wf'^d = wf^d - d_{j_d} \tag{f1.12}$$

$$vf'^d = vf^d - t^d_{i_d j_d} \tag{f1.13}$$

$$uf' = uf \cup \{j_d\} \cup Uf_{r'} \tag{f1.14}$$

$$sf'^{SR_l} = \begin{cases} sf^{SR_l} + \frac{1}{2} & \text{if } j_d \in S(l) \\ sf^{SR_l} + \frac{1}{2} - 1 & \text{if } j_d \in S(l) \text{ and } sf^{SR_l} + \frac{1}{2} \ge 1, l = 1, \dots, n_S \\ sf^{SR_l} & \text{otherwise} \end{cases} \tag{f1.15}$$

$$cf' = cf + c^d t^d_{i_d j_d} - \frac{(\pi_{i_d} + \pi_{j_d})}{2} - \sum_{l=1,\dots,n_S} ef_l \zeta_{S(l)} \tag{f1.16}$$

Here, $ef_l, l = 1, \dots, n_S$, is a binary parameter equal to 1 if and only if $j_d \in S(l)$ and $sf^{SR_l} + \frac{1}{2} \ge 1$. If $wf'^d \ge 0$, $\min\{t^d_{j_d,i}, i \in N^- \setminus uf\} \le vf'^d$, label $lf_{r'}$ is added to the label set $Lf_{i_t j_d}$. Note that sf'^{SR_l} denotes the state of SR inequality SR_l along partial synthetic-route r',

which is updated from sf^{SR_l} after the visit of the drone to node j_d. Hence, $sf^{'SR_l}$ first receives the value sf^{SR_l}, then it is increased by $\frac{1}{2}$ if $j_d \in S(l)$, and is decreased by 1 if it becomes no less than 1. Moreover, when $sf^{SR_l} + \frac{1}{2} \geq 1$, implying that the coefficient of the master variable corresponding to the complete synthetic-route propagated from partial synthetic-route r' in the SR inequality SR_l is 1, so $-\zeta_{S(l)}$ should be added to the reduced cost.

Subcase f1.3: The truck continues to serve another customer, and the truck and drone rejoin at the corresponding node. In this subcase, for each node $j_t \in N^- \setminus uf$, a new label $lf_{r'} = (tf^{'t}, tf^{'d}, wf^{'t}, wf^{'d}, vf^{'d}, uf^{'}, (sf^{'SR_l})_{l=1,\ldots,n_S}, cf^{'})$ is generated, where $wf^{'d}$ and $vf^{'d}$, are defined in (f1.4) and (f1.5), respectively, and

$$tf^{'t} = \max\{tf^t, e_{i_t}\} + t^t_{i_t j_t} \tag{f1.17}$$

$$tf^{'d} = \max\{tf^d, e_{i_d}\} + t^d_{i_d j_t} \tag{f1.18}$$

$$wf^{'t} = wf^t - d_{j_t} + wf^d \tag{f1.19}$$

$$uf^{'} = uf \cup \{j_t\} \cup Uf_{r'} \tag{f1.20}$$

$$sf^{'SR_l} = \begin{cases} sf^{SR_l} + \frac{1}{2} & \text{if } j_t \in S(l) \\ sf^{SR_l} + \frac{1}{2} - 1 & \text{if } j_t \in S(l) \text{ and } sf^{SR_l} + \frac{1}{2} \geq 1, l = 1, \ldots, n_S \\ sf^{SR_l} & \text{otherwise} \end{cases} \tag{f1.21}$$

$$cf^{'} = cf + c^t t^t_{i_t j_t} + c^d t^d_{i_d j_t} - \frac{(\pi_{i_t} + \pi_{i_d} + \pi_{j_t})}{2} - \sum_{l=1,\ldots,n_S} ef_l \zeta_{S(l)} \tag{f1.22}$$

If $wf^{'t} \geq 0$, $t^d_{j_t i_d} \leq vf$ and $\max\{\max\{tf^{'t}, e_{j_t}\}, tf^{'d}\} + t^t_{j_t,n+1} \leq L^t$(the truck route duration constraint), label $lf_{r'}$ is added to the label set $Lf_{j_t j_t}$.

Subcase f1.4: The truck and drone independently serve two other customers. In this subcase, for each pair of nodes $(j_t, j_d) \in (N^- \setminus uf, (D \cup \{n+1\}) \setminus uf)$, a new label $lf_{r'} = (tf^{'t}, tf^{'d}, wf^{'t}, wf^{'d}, vf^{'d}, uf^{'}, (sf^{'SR_l})_{l=1,\ldots,n_S}, cf^{'})$ is generated, where $tf^{'t}$, $tf^{'d}$, $wf^{'d}$, and $vf^{'d}$ are defined in (f1.17), (f1.10), (f1.12), and (f1.13), respectively, and

$$wf^{'t} = wf^t - d_{j_t} \tag{f1.23}$$

$$uf^{'} = uf \cup \{j_t, j_d\} \cup Uf_{r'} \tag{f1.24}$$

$$sf^{'SR_l} = \begin{cases} sf^{SR_l} + \frac{1}{2} & \text{if } j_t \text{ or } j_d \in S(l) \\ sf^{SR_l} + \frac{1}{2} - 1 & \text{if } j_t \text{ or } j_d \in S(l) \text{ and } sf^{SR_l} + \frac{1}{2} \geq 1, l = 1, \ldots, n_S \\ sf^{SR_l} & \text{otherwise} \end{cases} \tag{f1.25}$$

$$cf^{'} = cf + c^t t^t_{i_t j_t} + c^d t^d_{i_d j_d} - \frac{(\pi_{i_t} + \pi_{j_t} + \pi_{i_d} + \pi_{j_d})}{2} - \sum_{l=1,\ldots,n_S} ef^{'}_l \zeta_{S(l)} \tag{f1.26}$$

Here, $ef^{'}_l, l = 1, \ldots, n_S$, is a binary parameter equal to 1 if and only if $j_t, j_d \in S(l)$ or (j_t or $j_d \in S(l)$ and $sf^{SR_l} + \frac{1}{2} \geq 1$). If $wf^{'t} \geq 0$, $wf^{'d} \geq 0$, $\min\{t_{i,j_d}, i \in N^- \setminus uf\} \leq vf^{'d}$ and $\max\{tf^{'t}, e_{j_t}\} + t^t_{j_t,n+1} \leq L^t$, label $lf_{r'}$ is added to $Lf_{j_t j_d}$.

Subcase f1.5: The truck continues to serve another customer, while the drone stays at node i_d. In this subcase, for each node $j_t \in N^- \setminus uf$, a new label $lf_{r'} = (tf^{'t}, tf^{'d}, wf^{'t}, wf^{'d}, vf^{'d}, uf^{'}, (sf^{'SR_l})_{l=1,\ldots,n_S}, cf^{'})$ is generated, where $tf^{'t}$, $wf^{'t}$, $uf^{'}$, and $sf^{'SR_l}$ are defined in (f1.17), (f1.23), (f1.20), and (f1.21), respectively, and

$$tf^{'d} = tf^d \tag{f1.27}$$

$$wf^{'d} = wf^d \tag{f1.28}$$

$$vf^{'d} = vfd \tag{f1.29}$$

$$cf^{'} = cf + c^t t^t_{i_t j_t} - \frac{(\pi_{i_t} + \pi_{j_t})}{2} - \sum_{l=1,\ldots,n_S} ef_l \zeta_{S(l)} \tag{f1.30}$$

If $wf^{'t} \geq 0$ and $\max\{tf^{'t}, e_{j_t}\} + t^t_{j_t,n+1} \leq L^t$, label $lf_{r'}$ is added to the label set $Lf_{j_t i_d}$.

Case f2: $i_t = i_d$, implying that the drone is carried on the truck. In this case, we further consider two subcases (see Fig. 4 for detail).

Subcase f2.1: The truck together with the drone serves the next customer. In this subcase, for each node $j_t \in N^- \setminus uf$, a new label $lf_{r'} = (tf^{'t}, tf^{'d}, wf^{'t}, wf^{'d}, vf^{'d}, uf^{'}, (sf^{'SR_l})_{l=1,\ldots,n_S}, cf^{'})$ is generated, where $wf^{'t}$, $wf^{'d}$, $vf^{'d}$, $uf^{'}$, and $sf^{'SR_l}$ are defined in (f1.23), (f1.28), (f1.29), (f1.20), and (f1.21), respectively, and

$$tf^{'t} = tf^{'d} = \max\{\max\{tf^t, e_{i_t}\}, tf^d\} + t^t_{i_t j_t} \tag{f2.1}$$

$$cf^{'} = cf + c^t t^t_{i_t j_t} - \frac{(\pi_{i_t} + \pi_{j_t})}{2} - \sum_{l=1,\ldots,n_S} ef_l \zeta_{S(l)} \tag{f2.2}$$

If $wf^{'t} \geq 0$ and $\max\{tf^{'t}, e_{j_t}\} + t^t_{j_t,n+1} \leq L^t$, label $lf_{r'}$ is added to the label set $Lf_{j_t j_t}$.

Case f2.2: The truck and drone independently serve two other customers. In this subcase, for each pair of nodes $(j_t, j_d) \in (N^- \setminus uf, (D \cup \{n+1\}) \setminus uf)$, a new label $lf_{r'} = (tf^{'t}, tf^{'d}, wf^{'t}, wf^{'d}, vf^{'d}, uf^{'}, (sf^{'SR_l})_{l=1,\ldots,n_S}, cf^{'})$ is generated, where $tf^{'t}$, $uf^{'}$ and $sf^{'SR_l}$ are defined in (f2.1), (f1.24), and (f1.25), respectively, and

$$tf^{'d} = \max\{tf^d, tf^t\} + t^d_{i_d j_d} \tag{f2.3}$$

$$wf^{'d} = Q^d - d_{j_d} \tag{f2.4}$$

$$vf^{'d} = L^d - t^d_{i_d j_d} \tag{f2.5}$$

$$wf^{'t} = wf^t - Q^d - d_{j_t} \tag{f2.6}$$

$$cf^{'} = cf + c^t t^t_{i_t j_t} + c^d t^d_{i_d j_d} - \frac{(\pi_{i_t} + \pi_{j_t} + \pi_{j_d})}{2} - \sum_{l=1,\ldots,n_S} ef^{'}_l \zeta_{S(l)} \tag{f2.7}$$

If $wf^{'t} \geq 0$, $\min\{t_{j_d,i}, i \in N^- \setminus uf\} \leq vf^{'d}$, and $\max\{tf^{'t}, e_{j_t}\} + t_{j_t,n+1} \leq L^t$ and is added to the label set $Lf_{j_t j_d}$.

Forward dominance rules. The performance of any labelling algorithm heavily depends on the dominance rules, which allow one to discard many labels that cannot lead to a better complete route over another label (or set of labels). The following dominance rule can be applied for our forward label extension.

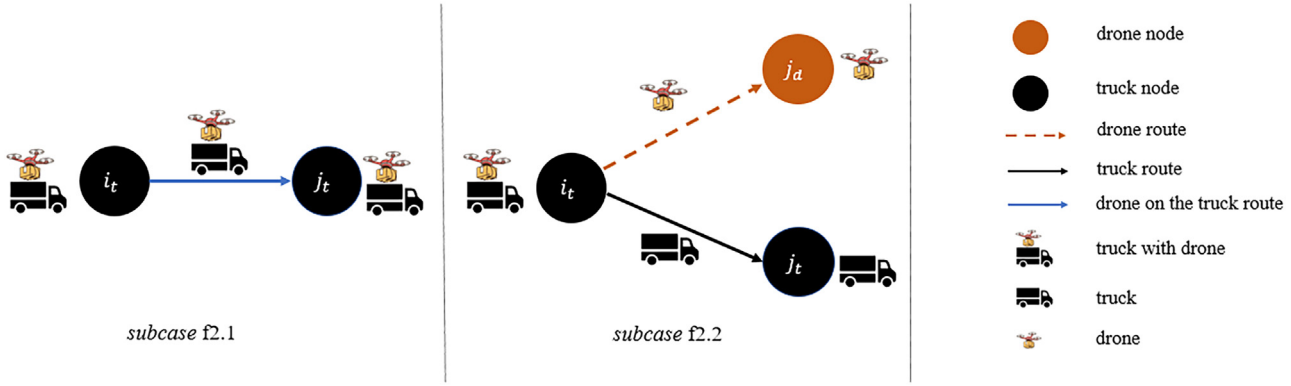

Fig. 4. The subcases of Case 2 in forward label extension.

Lemma 1. *For any two forward labels* $lf_r = (tf^t, tf^d, wf^t, wf^d, vf^d, uf, (sf^{SR_l})_{l=1,\dots,n_S}, cf)$ *and* $lf_{r'} = (tf^{'t}, tf^{'d}, wf^{'t}, wf^{'d}, vf^{'d}, uf^{'}, (sf^{'SR_l})_{l=1,\dots,n_S}, cf^{'})$, *the former dominates the latter if*

(i) $tf^t \le tf^{'t}$
(ii) $tf^d \le tf^{'d}$
(iii) $wf^t \ge wf^{'t}$
(iv) $wf^d \ge wf^{'d}$
(v) $vf^d \ge vf^{'d}$
(vi) $uf \subseteq uf^{'}$
(vii) $uf \cap D \subseteq uf^{'} \cap D$
(viii) $cf - \sum_{l:sf^{SR_l} > sf^{'SR_l}} \zeta_{S(l)} \le cf^{'}$

Conditions (i)–(v) give the dominant conditions in terms of time, load, and battery power, respectively. Conditions (vi) and (vii) respectively represent the dominant relationship between sets that can be accessed in subsequent expansion. Condition (viii) denotes the reduced cost relationship of the two labels.

Moreover, for any label, if the lower bound on the reduced cost of any complete synthetic-route extending from the partial synthetic-route corresponding to this label is no less than zero, it is evident that this label can be discarded. The following lemma shows how to obtain a lower bound on a forward label.

Lemma 2. *For any forward label* $lf_r = (tf^t, tf^d, wf^t, wf^d, vf^d, uf, (sf^{SR_l})_{l=1,\dots,n_S}, cf)$ *in the label set* $Lf_{i_t i_d}$,

$$LB(lf_r) = \begin{cases} cf - \frac{\pi_{i_t}+\pi_{i_d}}{2} - \frac{F+\sigma}{2} + \sum_{j\in N\setminus uf} \min\{0, -\pi_j\} + c^t t^t_{i_t,n+1} & \text{if } i_t \neq i_d \\ cf - \frac{\pi_{i_t}}{2} - \frac{F+\sigma}{2} + \sum_{j\in N\setminus uf} \min\{0, -\pi_j\} + c^t t^t_{i_t,n+1} & \text{otherwise} \end{cases}$$

is a lower bound on the reduced cost of any complete synthetic-route extending from the partial synthetic-route r, and the label lf_r *can be discarded if* $LB(lf_r) \ge 0$.

Proof. Let $r \oplus r^*$ be any feasible complete synthetic-route extending from r along the partial synthetic-route r^* and $N(r^*)$ be the set of customers served in r^*. It is clear that $-\frac{F+\sigma}{2} + \sum_{j\in N\setminus uf} \min\{0, -\pi_j\}$ is a lower bound on the contributions of the nodes in $N(r^*) \cup \{n+1\}$ to the reduced cost, as required. □

The backward labelling algorithm

Backward label definition. For each pair of $(j_t, j_d) \in N \times N$, let $Lb_{j_t j_d}$ be the set of forward labels, each of which encodes a partial synthetic-route that extends backwardly from the depot $n+1$ to its predecessor and ends at nodes j_t and j_d. Each label $lb_r = (tb^t, tb^d, wb^t, wb^d, vb^d, ub, (sb^{SR_l})_{l=1,\dots,n_S}, cb) \in Lb_{j_t j_d}$ is composed of the following semantics:

tb^t: the latest departure time of the truck from node j_t
tb^d: the latest departure time of the drone from node j_d
wb^t: the cumulative load of customer demand of the truck-drone combination before serving node j_t excluding the cumulative load of customer demand of the drone after its latest take-off if it is not on the truck
wb^d: the cumulative load of customer demand of the drone after its latest take-off before serving node j_d
vb^d: the cumulative flight time of the drone after its latest take-off before serving node j_d and ub, sb^{SR_l}, and cb have the same meaning as uf, sf^{SR_l}, and cf, respectively.

We say that a node i is unreachable from the partial synthetic-route r if one of the following conditions is satisfied:

(i) $wb^t + wb^d + d_i + d_{j_d} + d_{j_t} > Q^t$;
(ii) $\min\{tb^t, l_{j_t}\} - t^t_{ij_t} < e_i$ if $i \in N^+ \setminus D$; and
(iii) $\min\{tb^t, l_{j_t}\} - t^t_{ij_t} < e_i$ and $\min\{tb^d, l_{j_d}\} - t^d_{ij_d} < e_i$ if $i \in D$.

Backward label extension

The label set $Lb_{n+1,n+1}$ is initialized with $(L^t, L^t, 0, 0, 0, \emptyset, \mathbf{0}, 0)$. For each label $lb_r = (tb^t, tb^d, wb^t, wb^d, vb^d, ub, (sb^{SR_l})_{l=1,\dots,n_S}, cb)$, to generate new states, there are two cases depending on whether the drone is separated from the truck to consider.

Case b1: $j_t \neq j_d$, implying that the drone is separated from the truck. In this case, we further consider four subcases (see Fig. 5 for detail).

Subcase b1.1: The truck continues to serve another customer, and the truck and drone rejoin at the corresponding node. In this subcase, for each $i_t \in N^+ \setminus ub$, a new label $lb_{r'} = (tb^{'t}, tb^{'d}, wb^{'t}, wb^{'d}, vb^{'d}, ub^{'}, (sb^{'SR_l})_{l=1,\dots,n_S}, cb^{'})$ is generated, where

$$tb^{'t} = \min\{tb^t, l_{j_t}\} - t^t_{i_t j_t} \quad \text{(b1.1)}$$

$$tb^{'d} = \min\{tb^d, l_{j_t}\} - t^d_{i_t j_d} \quad \text{(b1.2)}$$

$$wb^{'t} = wb^t + d_{j_t} + wb^d + d_{j_d} \quad \text{(b1.3)}$$

$$wb^{'d} = 0 \quad \text{(b1.4)}$$

$$vb^{'d} = 0 \quad \text{(b1.5)}$$

$$ub^{'} = uf \cup \{i_t\} \cup Ub_{r'} \quad \text{(b1.6)}$$

$$sb^{'SR_l} = \begin{cases} sb^{SR_l} + \frac{1}{2} & \text{if } i_t \in S(l) \\ sb^{SR_l} + \frac{1}{2} - 1 & \text{if } i_t \in S(l) \text{ and } sb^{SR_l} + \frac{1}{2} \ge 1, l=1,\dots,n_S \\ sb^{SR_l} & \text{otherwise} \end{cases} \quad \text{(b1.7)}$$

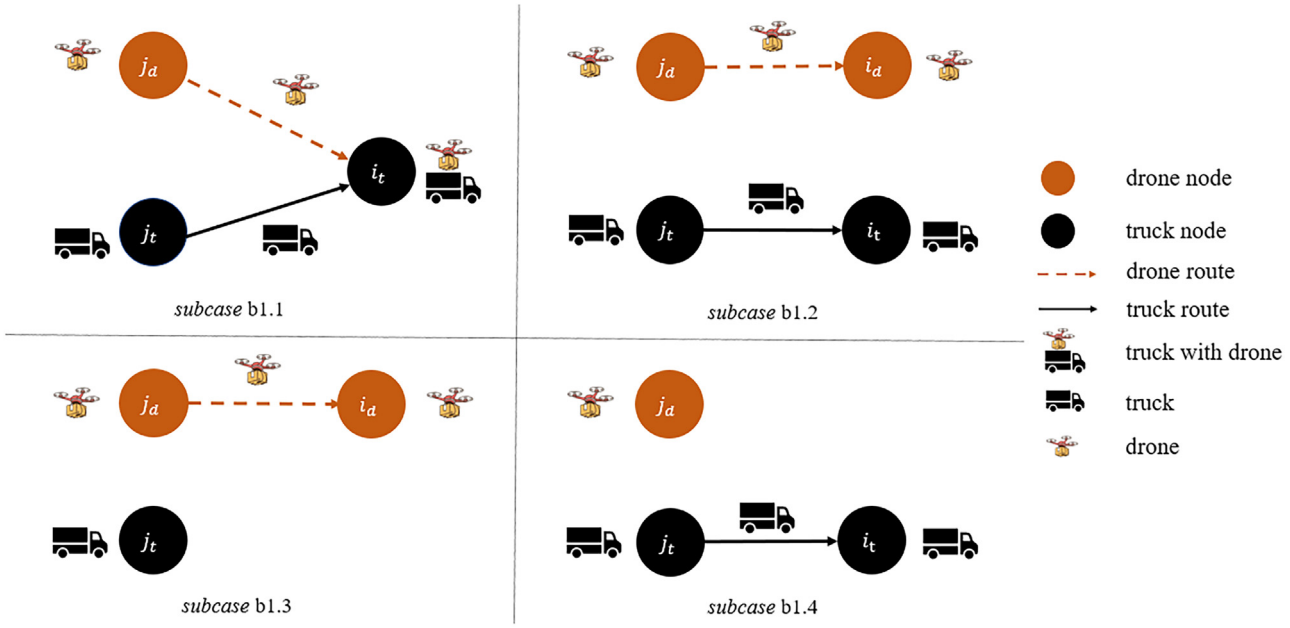

Fig. 5. The subcases of Case 1 in backward label extension.

$$cb' = cb + c^t t^t_{i_t j_t} + c^d t^d_{i_t j_d} - \frac{(\pi_{i_t} + \pi_{j_t} + \pi_{j_d})}{2} - \sum_{l=1,\ldots,n_S} eb_l \zeta_{S(l)} \tag{b1.8}$$

Here $Ub_{r'}$ is the subset of the nodes in $N^+ \setminus ub$ that are unreachable from the partial synthetic-route r', and $eb_l, l = 1, \ldots, n_S$ is defined as ef_l. If $wb'^t + d_{i_t} \le Q^t$, $vb^d + t_{j_d i_t} \le L^d$, and $\min\{\min\{tb'^t, l_{i_t}\}, tb'^d\} - t^t_{0 i_t} \ge 0$, label $lb_{r'}$ is added to the label set $Lb_{i_t i_t}$.

Subcase b1.2: The truck and drone independently serve two other customers. In this subcase, for each pair of nodes $(i_t, i_d) \in (i_t \in N^+ \setminus ub, (D \cup \{0\}) \setminus ub)$, a new label $lb_{r'} = (tb'^t, tb'^d, wb'^t, wb'^d, vb'^d, ub', (sb'^{SR_l})_{l=1,\ldots,n_S}, cb')$ is generated, where tb'^t and tb'^d are defined in (b1.1), (b1.2), respectively, and

$$wb'^t = wb^t + d_{j_t} \tag{b1.9}$$

$$wb'^d = wb^d + d_{j_d} \tag{b1.10}$$

$$vb'^d = vb^d + t^d_{i_d j_d} \tag{b1.11}$$

$$ub' = ub \cup \{i_t, i_d\} \cup Ub_{r'} \tag{b1.12}$$

$$sb'^{SR_l} = \begin{cases} sb^{SR_l} + \frac{1}{2} & \text{if } i_t \text{ or } i_d \in S(l) \\ sb^{SR_l} + \frac{1}{2} - 1 & \text{if } i_t \text{ or } i_d \in S(l) \text{ and } sb^{SR_l} + \frac{1}{2} \ge 1, l = 1, \ldots, n_S \\ sb^{SR_l} & \text{otherwise} \end{cases} \tag{b1.13}$$

$$cb' = cb + c^t t^t_{i_t j_t} + c^d t^d_{i_d j_d} - \frac{(\pi_{i_t} + \pi_{j_t} + \pi_{i_d} + \pi_{j_d})}{2} - \sum_{l=1,\ldots,n_S} eb'_l \zeta_{S(l)} \tag{b1.14}$$

Here, $eb'_l, l = 1, \ldots, n_S$ is defined as ef'_l. If $wb'^t + wb'^d + d_{i_t} + d_{i_d} \le Q^t$, $wb'^d + d_{i_d} \le Q^d$, $vb'^d + \min\{t_{i,i_d}, i \in N^+ \setminus ub\} \le L^d$, and $\min\{tb'^t, l_{i_t}\} - t_{0 i_t} \ge 0$, label $lb_{r'}$ is added to the label set $Lb_{i_t i_d}$.

Subcase b1.3: The drone continues to serve another customer, and the truck stays at node j_t. In this subcase, for each $i_d \in (D \cup \{0\}) \setminus ub$, a new label $lb_{r'} = (tb'^t, tb'^d, wb'^t, wb'^d, vb'^d, ub', (sb'^{SR_l})_{l=1,\ldots,n_S}, cb')$ is generated, where tb'^d, wb'^d, and vb'^d are defined in (b1.2), (b1.10), and (b1.11), respectively, and

$$tb'^t = tb^t \tag{b1.15}$$

$$wb'^t = wb^t \tag{b1.16}$$

$$ub' = ub \cup \{i_d\} \cup Ub_{r'} \tag{b1.17}$$

$$sf'^{SR_l} = \begin{cases} sb^{SR_l} + \frac{1}{2} & \text{if } i_d \in S(l) \\ sf^{SR_l} + \frac{1}{2} - 1 & \text{if } i_d \in S(l) \text{ and } sb^{SR_l} + \frac{1}{2} \ge 1, l = 1, \ldots, n_S \\ sb^{SR_l} & \text{otherwise} \end{cases} \tag{b1.18}$$

$$cb' = cb + c^d t^d_{i_d j_d} - \frac{(\pi_{i_d} + \pi_{j_d})}{2} - \sum_{l=1,\ldots,n_S} eb_l \zeta_{S(l)} \tag{b1.19}$$

If $wb'^d + d_{i_d} \le Q^d$ and $vb'^d + \min\{t_{i,i_d}, i \in N^+ \setminus ub\} \le L^d$, label lb_r' is added to the label set $Lb_{j_t i_d}$.

Subcase b1.4: The truck continues to serve another customer, while the drone stays at node j_d. In this subcase, for each $i_t \in N^+ \setminus ub$, a new label $lb_{r'} = (tb'^t, tb'^d, wb'^t, wb'^d, vb'^d, ub', (sb'^{SR_l})_{l=1,\ldots,n_S}, cb')$ is generated, where tb'^t, wb'^t, ub', and sb'^{SR_l} are defined in (b1.1),(b1.9), (b1.6), and (b1.7), respectively, and

$$tb'^d = tb^d \tag{b1.20}$$

$$wb'^d = wb^d \tag{b1.21}$$

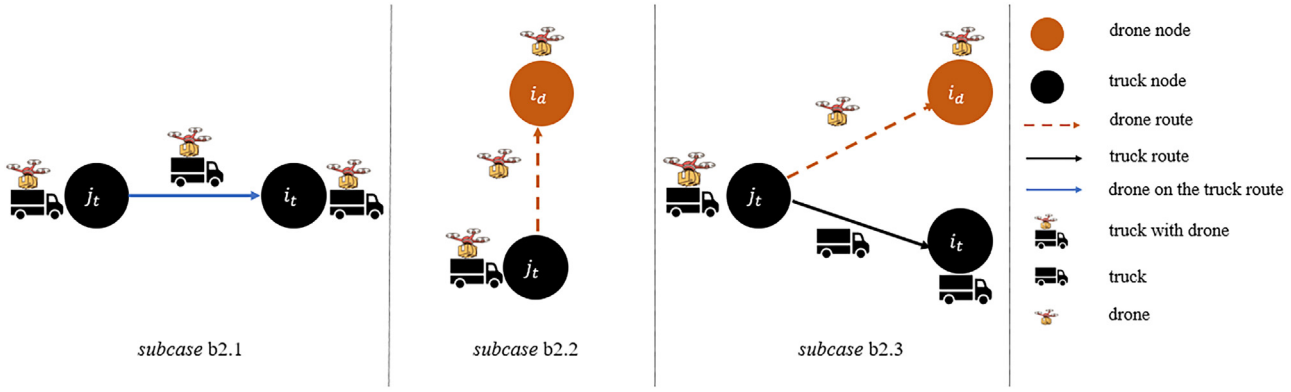

Fig. 6. The subcases of Case 2 in backward label extension.

$$vb'^{d} = vb^{d} \tag{b1.22}$$

$$cb' = cb + c^{t}t^{t}_{i_t j_t} - \frac{(\pi_{i_t} + \pi_{j_t})}{2} - \sum_{l=1,\dots,n_S} eb_l \zeta_{S(l)} \tag{b1.23}$$

If $wb'^{t} + d_{i_t} \le Q^{t}$ and $\min\{tb'^{t}, l_{i_t}\} - t^{t}_{0i_t} \ge 0$, label $lb_{r'}$ is added to the label set $Lb_{i_t j_d}$.

Case b2: $j_t = j_d$, implying that the drone is carried on the truck. In this case, we further consider three subcases (see Fig. 6 for detail).

Subcase b2.1: The truck together with the drone serves the next customer. In this subcase, for each node $j_t \in N^{+} \setminus ub$, a new label $lb_{r'} = (tb'^{t}, tb'^{d}, wb'^{t}, wb'^{d}, vb'^{d}, ub', (sb'^{SR_l})_{l=1,\dots,n_S}, cb')$ is generated, where wb'^{t}, wb'^{d}, vb'^{d}, ub', and sb'^{SR_l} are defined in (b1.9), (b1.21), (b1.22), (b1.6), and (b1.7), respectively, and

$$tb'^{t} = tb'^{d} = \min\{\min\{tb^{t}, l_{j_t}\}, tb^{d}\} - t^{t}_{i_t j_t} \tag{b2.1}$$

$$cb' = cb + c^{t}t^{t}_{i_t j_t} - \frac{(\pi_{i_t} + \pi_{j_t})}{2} - \sum_{l=1,\dots,n_S} eb_l \zeta_{S(l)} \tag{b2.2}$$

If $wb'^{t} + d_{i_t} \le Q^{t}$ and $\min\{tb'^{t}, l_{i_t}\} - t^{t}_{0i_t} \ge 0$, label $lb_{r'}$ is added to the label set $Lb_{i_t i_t}$.

Subcase b2.2: The drone serves another customer and the truck stay at j_t. In this subcase, for each node $i_d \in (D \cup \{0\}) \setminus ub$, a new label $lb_{r'} = (tb'^{t}, tb'^{d}, wb'^{t}, wb'^{d}, vb'^{d}, ub', (sb'^{SR_l})_{l=1,\dots,n_S}, cb')$ is generated, where tb'^{t}, wb'^{t}, wb'^{d}, ub',and sb'^{SR_l}, are defined in (b1.15), (b1.16), (b1.10), (b1.17), and (b1.18), respectively, and

$$tb'^{d} = \min\{tb^{d}, tb^{t}\} - t^{d}_{i_d j_d} \tag{b2.3}$$

$$vb'^{d} = t^{d}_{i_d j_d} \tag{b2.4}$$

$$cb' = cb + c^{d}t^{d}_{i_d j_d} - \frac{\pi_{i_d}}{2} - \sum_{l=1,\dots,n_S} eb_l \zeta_{S(l)} \tag{b2.5}$$

If $vb'^{d} + \min\{t_{i,i_d}, i \in N^{+} \setminus ub\} \le L^{d}$, label $lb_{r'}$ is added to the label set $Lb_{j_t i_d}$.

Subcase b2.3: The truck and drone independently serve two other customers. In this subcase, for each pair of nodes $(i_t, i_d) \in (N^{+} \setminus ub, (D \cup \{0\}) \setminus ub)$, a new label $lb_{r'} = (tb'^{t}, tb'^{d}, wb'^{t}, wb'^{d}, vb'^{d}, ub', (sb'^{SR_l})_{l=1,\dots,n_S}, cb')$ is generated, where tb'^{d}, wb'^{t}, wb'^{d}, vb'^{d}, ub', and sb'^{SR_l} are defined in (b2.3), (b1.9), (b1.21), (b2.4), (b1.12), and (b1.13), respectively, and

$$tb'^{t} = \min\{tb^{d}, \min\{tb^{t}, l_{j_t}\}\} - t^{t}_{i_t j_t} \tag{b2.6}$$

$$cb' = cb + c^{t}t^{t}_{i_t j_t} + c^{d}t^{d}_{i_d j_d} - \frac{(\pi_{i_t} + \pi_{j_t} + \pi_{i_d})}{2} - \sum_{l=1,\dots,n_S} eb'_l \zeta_{S(l)} \tag{b2.7}$$

If $t^{d}_{i_d j_d} + \min\{t_{i,i_d}, i \in N^{+} \setminus ub\} \le L^{d}$, $wb'^{t} + d_{i_t} + d_{i_d} \le Q^{t}$, $\min\{\min\{tb'^{t}, l_{i_t}\}, \min\{tb'^{d}, l_{i_d}\} - t^{d}_{i_t i_d}\} - t^{t}_{0i_t} \ge 0$, and $wb'^{d} \le Q^{d}$, label $lb_{r'}$ is added to label set $Lb_{i_t i_d}$.

Backward dominance rules Analogous to the forward labelling algorithm, the following results hold.

Lemma 3. *For any two forward labels* $lb_r = (tb^{t}, tb^{d}, wb^{t}, wb^{d}, vb^{d}, ub, (sb^{SR_l})_{l=1,\dots,n_S}, cb)$ *and* $lb_{r'} = (tb'^{t}, tb'^{d}, wb'^{t}, wb'^{d}, vb'^{d}, ub', (sb'^{SR_l})_{l=1,\dots,n_S}, cb')$ *in the label set* $Lb_{i_t i_d}$, *the former dominates the latter if*

(i) $tb^{t} \ge tb'^{t}$
(ii) $tb^{d} \ge tb'^{d}$
(iii) $wb^{t} \le wb'^{t}$
(iv) $wb^{d} \le wb'^{d}$
(v) $vb^{d} \le vb'^{d}$
(vi) $ub \subseteq ub'$
(vii) $ub \cap D \subseteq ub' \cap D$
(viii) $cb - \sum_{l:sb^{SR_l} > sb'^{SR_l}} \zeta_{S(l)} \le cb'$

Lemma 4. *For any backward label* $lb_r = (tb^{t}, tb^{d}, wb^{t}, wb^{d}, vb^{d}, ub, (sb^{SR_l})_{l=1,\dots,n_S}, cb)$ *in the label set* $Lb_{j_t j_d}$,

$$LB(lb_r) = \begin{cases} cb - \frac{\pi_{j_t} + \pi_{j_d}}{2} - \frac{F+\sigma}{2} + \sum_{j \in N \setminus ub} \min\{0, -\pi_j\} + c^{t}t^{t}_{0j_t} & \text{if } j_t \ne j_d \\ cb - \frac{\pi_{j_t}}{2} - \frac{F+\sigma}{2} + \sum_{j \in N \setminus ub} \min\{0, -\pi_j\} + c^{t}t^{t}_{0j_t} & \text{otherwise} \end{cases}$$

is a lower bound on the reduced cost of any complete synthetic-route extending from the partial synthetic-route r, and the label lb_r *can be discarded if* $LB(lb_r) \ge 0$.

Concatenation of forward and backward labels

In our bounded bidirectional labelling algorithm, we take time as the crucial resource and only extend forward and backward labels such that $tf^{t} < \frac{L^{t}}{2}$ and $tb^{t} \ge \frac{L^{t}}{2}$. Any forward label $lf_r \in Lf_{i_t i_d}$ and backward label $lb_r \in Lb_{i_t i_d}$ can be connected if the following conditions are satisfied:

(i) $N(lf_r) \cap N(lb_r) = \{i_t, i_d\}$
(ii) $\max\{tf^{t}, e_{i_t}\} \le tb^{t}$
(iii) $\max\{tf^{d}, e_{i_d}\} \le tb^{d}$
(iv) $wf^{t} \ge wb^{t}$
(v) $wf^{d} \ge wb^{d}$
(vi) $vf^{d} \ge vb^{d}$

Here $N(lf_r)$ and $N(lb_r)$ denote the sets of nodes visited by the partial synthetic-routes corresponding to labels lf_r and lb_r, respectively.

4.3. Branching strategy

When a solution to the current RLMP at a node is fractional and no SR inequalities can be found, branching is required. Let $\bar{\lambda}_r, r \in R_s$ be the solution to the current RLMP and let $\bar{x}_{ij} = \sum_{r\in\bar{R}_s} b_{ij}^r \bar{\lambda}_r$, where b_{ij}^r is a binary parameter equal to 1 if and only if arc (i, j) is traversed by the synthetic-route r. We explore the following three-stage hierarchical branching strategy.

(i) Branching on the number of vehicles. If the sum of the number of vehicles in the current solution is fractional, we create two branches $\sum_{r\in R_s} \lambda_r \le \left\lceil \sum_{r\in\bar{R}_s} \bar{\lambda}_r \right\rceil$ and $\sum_{r\in R_s} \lambda_r \ge \left\lceil \sum_{r\in\bar{R}_s} \bar{\lambda}_r \right\rceil$. For the dual variable of this constraint, we can easily integrate it with the dual variable of the master problem constraint (39) and pass it to the subproblem without affecting the structure of the subproblem.

(ii) Branching on the outflow of a node set S. We choose a set $S \subseteq C$ with size two whose outflow $x(\delta^+(S)) = \sum_{r\in\bar{R}_s} \sum_{i\in S} \sum_{j\in C\setminus S} b_{ij}^r \bar{\lambda}_r$ is closest to 1.5, and create two branches $\sum_{r\in R_s} \sum_{i\in S} \sum_{j\in C\setminus S} b_{ij}^r \lambda_r \le 1$ and $\sum_{r\in R_s} \sum_{i\in S} \sum_{j\in C\setminus S} b_{ij}^r \lambda_r \ge 2$.

(iii) Branching on an individual arc variable x_{ij}. We choose an arc (i, j) such that $\bar{x}_{ij}$ is closest to 0.5, and create two branches $\bar{x}_{ij} = 0$ and $\bar{x}_{ij} = 1$.

When arc (i, j) is forbidden at a node of the search tree, i.e., the trucks and drones cannot traverse arc (i, j), we can remove this arc from the directed graph corresponding to this node and discard any columns using it. On the other hand, when arc (i, j) is fixed as 1, we need to eliminate all the ingoing arcs with the ending node j and all the outgoing arcs with the starting node i from the directed graph, and discard any columns that do not traverse arc (i, j) but visit node i or j.

4.4. Enhancement strategies

In this section we develop some enhancement strategies to improve the performance of the developed BPC algorithm, including strategies to accelerate the solution for the pricing problem and strategies to obtain high-quality feasible solutions.

4.4.1. Strategies for solving pricing problem

The CCG algorithm has a significant drawback that one has to solve the strongly *NP*-hard pricing problem several times. Our numerical studies demonstrate that most of the computational time of the BPC algorithm is consumed in solving the pricing problem. So one way to improve the performance of the algorithm is to heuristically solve the pricing problem, given that only in the last iteration of the CCG algorithm an exact solution is needed to ensure optimality.

In the following we design three heuristic strategies, namely greedy heuristics, Tabu search, and dynamic *ng*-route relaxation. Four algorithms are, thus, available to generate columns, i.e., the greedy heuristics, Tabu search, dynamic *ng*-route relaxation, and bounded bidirectional labelling algorithm, which are executed in sequence in each column generation iteration. Specifically, once columns with negative reduced costs are found by an algorithm, they are added to the current RLMP, which is re-optimized to start a new column generation iteration. Otherwise, the subsequent algorithm is executed.

It is worth noting that we only apply the greedy heuristics and Tabu search to find the truck routes with low reduced costs. Based on each obtained truck route, we try to find better synthetic-routes by letting the drone visit some nodes in the obtained truck route or outer nodes.

Greedy heuristics. We develop two greedy heuristics, namely deterministic greedy heuristic and randomized greedy heuristic, to quickly solve the pricing problem. In the developed heuristics, we explore the nearest neighbour strategy to construct a solution by extending arcs to a partial route starting from the origin depot and going forward, or starting from the end depot and going backward. In the deterministic version, in each iteration the most valuable arc among those that will not form cycles is extended. In the randomized version, however, in each iteration the next arc is randomly chosen among the five most valuable arcs, and the algorithm is repeated several times to generate the best feasible solution.

Specifically, our procedure first executes the deterministic greedy heuristic. If it cannot find columns with negative reduced costs, we invoke the randomized greedy heuristic.

Tabu search Tabu search is a heuristic method proposed by Gendreau et al. (1994), and then widely used to solve the pricing problem (Desaulniers et al., 2008; Dayarian et al., 2015). We use the basic columns with zero reduced cost in the solution to the current RLMP as the input of this algorithm, based on which it may be easy to find the routes with negative reduced costs. The algorithm works independently on each of these routes. In each iteration, the search moves to the best feasible neighbouring solution in terms of the following two kinds of operators: (i) remove a node from the current route and (ii) insert a node into the current route in all the possible insertion positions.

Dynamic ng*-route relaxation*. The *ng*-route relaxation, allowing possibly cyclic *ng*-routes introduced by Baldacci et al. (2011), requires the definition of neighbourhood of each node $i \in C$, i.e., $NG_i \subseteq C$ with $i \in NG_i$. Specifically, an *ng*-route is a route where a node i is allowed to be visited more than once only if between repeat visits it also visits at least one other node j such that $i \notin NG_j$. In other words, returning to the same node i is allowed if the cycle first leaves its neighbourhood NG_i.

In the process of *ng*-route expansion, we regard the current truck node and drone node as a whole, and jointly consider the current *ng*-list. We need to change the updating rule of resource uf in our bounded bidirectional labeling algorithm. For example, (f1.5) should be replaced by $uf' = (NG_j \setminus uf) \cup Uf_{r'}$.

Clearly, the number of neighbours of a node is crucial. Generally, the smaller the neighbourhood, the weaker are the obtained lower bounds of the MP, and the less is the computational time. To efficiently adjust the neighbourhoods, we explore the method based on dynamical augmentation of the ng neighbourhoods introduced by Roberti & Mingozzi (2014). Specifically, we begin with a small or empty neighbourhood NG_i and execute the corresponding *ng*-route relaxation algorithm to solve the pricing problem. If a node i is visited more than once in the optimal *ng*-route obtained, which occurs in the cycle(s) containing node i, we add the node to the neighbourhood of all nodes in the cycle(s).

4.4.2. Recovering a feasible solution

Finding a good feasible solution to the original problem is an essential component of the branch-and-bound algorithm. A good upper bound can prune more redundant nodes, so accelerating the convergence of the algorithm. At each fractional node in our search tree, we deploy a heuristic to construct a “good” feasible solution.

The heuristic appends all the columns in the incumbent to the final RLMP at the node, and removes the columns whose reduced costs are larger than the difference between the current upper bound and low bound. We solve the resulting RLMP with integrity constraints, called sub-RLMP, using CPLEX. This gives a better chance to obtain a better feasible solution. The resulting sub-

ILP may also be potentially large and difficult to solve, so its exploration must often be truncated. We do so by setting a time limit.

5. Numerical studies

In this section we present extensive numerical studies to: (i) evaluate the efficiency of the enhancement strategies and the effectiveness of the developed BPC algorithm, (ii) investigate the impacts of the key model parameters on the algorithm performance and solution structure, and (iii) demonstrate the gain of the truck-based drone delivery over the truck-only delivery.

We coded all the algorithms in Java and solved the linear programs using CPLEX 12.8. We conducted all the computational runs on a PC equipped with a 64-bit 2.4 gigahertz Intel core processor with 8 gigabyte of RAM.

5.1. Instance generation

As far as we know, no standard test datasets are available for the problem under study. So we adapted the method in Wang & Sheu (2019) to generate our problem instances. We randomly chose the locations within an area of 15 square kilometers. We generated three types of instances. For the first two types, we randomly and independently selected the x and y coordinates of customer locations from $\{0, 0.5, 1, 1.5, \ldots, 15\}$. We applied the above rule to select the coordinates of the depot of the first type of instances, and took the mean values of the coordinates of customer locations as the coordinates of the depot of the second type of instances. For the third type of instances, we generated the x and y coordinates of customer locations in a clustering manner, which equal $\gamma\cos(\varphi)$ and $\gamma\sin(\varphi)$, respectively, where γ denotes a distance randomly sampled from the normal distribution $N(0, 10^2)$ and φ is an angle randomly sampled from the uniform distribution $U[0, 2\pi]$, and the depot is located at the origin of the $x-y$ axes.

For each instance type, we generated instances of different sizes with $n \in \{10, 20, 30, 35, 40, 45\}$. For each combination of instance type and n, we generated three instances by varying the coordinates of customer locations or the depot. Let ϑ be the ratio of customers that can be served by drones. For each instance with n customers, we generated the customer subset D with $|D| = \lceil n\vartheta \rceil$, where each customer was randomly chosen from the n customers. To perform extensive analysis, we consider the cases with $\vartheta \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$. Thus, we have 270 instances in total.

For each instance, we used the method introduced in Solomon (1987) to generate the time windows of customers. According to the working time regulations, we set the maximum duration of the truck route L^t to 480 minutes, and thus set the time window of the depot to $[0, 480]$. For each customer, we randomly generated the centre of the time window according to Solomon's method, assuming that half the width of the time window follows the normal distribution $N(\mu, \chi^2)$ with $\mu = 30$ and $\chi = 10$.

Similar to Wang & Sheu (2019), we set the maximum carrying capacities of the trucks and drones Q^t and Q^d to 100 and 20 weight units, respectively, and generated customer demands randomly and independently from $\{10, 20, 30, 40, 50\}$ weight units. We set the maximum travel time of the drone with a fully charged battery L^d to 30 minutes, and set both the speeds of the truck and drone v^t and v^d to 40 kilometers per hour, implying that $\alpha = 1$. Following Agatz et al. (2018), we set $\beta = 2$, implying that $t^d_{ij} = \frac{1}{2}t^t_{ij}$ for any arc $(i, j) \in A$.

We set the cost parameters as follows: fixed assignment cost $F = \$20$, truck travel cost $c^t = \$0.083$ per minute (fuel consumption is about 0.15 liter/km for a truck and oil cost is about \$0.83 per liter, see Global Petrol Prices (2018), and drone travel cost $c^d = \$0.021$ per minute (a drone generally costs \$0.05 per mile, see Kim (2016)).

5.2. Effectiveness of the enhancement strategies and SR inequalities

In this section we focus on testing the effectiveness of enhancement strategies presented in Section 4.4.1 and the SR inequalities using the instances with 35 customers under different ϑ.

In all the tables, the columns "No. Uns", "Ave. Time", "Ave. Col", "Ave. Cut", and "Ave. Gap" give the numbers of instances that cannot be solved within a predefined time limit, the average computational time (in seconds) for the solved instances only, the average numbers of columns and cuts generated, and the average percentage gap between the best upper and lower bounds for the unsolved instances prior to termination, respectively. The row "Ave/To" provides the average value or total number of the corresponding terms.

5.2.1. Effectiveness of the enhancement strategies

To assess the effectiveness of the enhancement strategies to accelerate the column generation process, we compare the following five versions of the algorithm for solving the LMP at the root node: (i) the column generation (CG) algorithm, i.e., the CCG algorithm without SR inequality generation and any enhancement strategy (baseline); (ii) the CG algorithm with the greedy heuristic (GH); (iii) the CG algorithm with Tabu search (TS); (iv) the CG algorithm with dynamic *ng*-route relaxation (NGR); and (v) the CG algorithm with all enhancement strategies (ALL). For each run of the comparison algorithms, we set a time limit of 1800 seconds.

We summarize the comparison results in Table 3, where we report the "No. Uns", "Ave. Time", "Ave. Col", the average percentage gap between the best upper and lower bounds of the LMP for the unsolved instances prior to termination ("Ave. rGap"), and the average number of times that the greedy heuristic ("N1"), Tabu search ("N2"), dynamic *ng*-route relaxation algorithm ("N3"), and exact labelling algorithm ("N0") are invoked, respectively.

The results clearly demonstrate that all the enhancement strategies improve the effectiveness of the CG algorithm and complement each other: more instances are solved and less average computational time is consumed for the solved instances. With respect to the marginal contribution to algorithmic performance, dynamic *ng*-route relaxation seems to be the strategy that produces the largest improvement, and the greedy heuristic and Tabu search preform at about the same level. In addition, compared with the single enhancement strategy, the combined use of the enhancement strategies produces the largest improvement in algorithmic performance.

Specifically, from the column "No. Uns", we see that the addition of all enhancement strategies enables the optimal solving 44 of the 45 the instances, whereas the addition of only the greedy heuristic, only Tabu search, only dynamic *ng*-route relaxation, and no enhancement strategy are capable of optimal solving only 43, 43, 43, and 42 instances, respectively. Perhaps more significantly, the values of "Ave. rGap" are considerably smaller when all enhancement strategies are added. Looking more closely at the average computational time, the addition of only the greedy heuristic, only Tabu search, only dynamic *ng*-route relaxation, and all enhancement strategies leads to a reduction of 28.74%, 29.49%, 44.92%, and 62.55% on average, respectively, against the baseline results. This can be explained by the fact that most of the computational time of the CCG algorithm is consumed in solving the pricing problem with the exact labeling algorithm. Indeed, column "N0" indicates that the average number of times that the exact labeling algorithm is invoked decreases by up to 28.67%, 39.26%, 52.69%, and 69.28% on average by the addition of only the greedy

Table 3
Performance of computational enhancements.

n	ϑ	Baseline					GH						TB					
		No. Uns	Ave. Time (seconds)	Ave. Col	N0	Ave. rGap	No. Uns	Ave. Time (seconds)	Ave. Col	N0	N1	Ave. rGap	No. Uns	Ave. Time (seconds)	Ave. Col	N0	N2	Ave. rGap
35	0.1	0	164.10	262.67	53.00	0	0	90.10	161.00	30.67	30.67	0	0	95.43	247.33	31.33	34.33	0
	0.3	0	389.28	311.67	62.67	0	0	265.30	226.00	38.67	34.67	0	0	196.77	271.67	36.33	30.33	0
	0.5	0	681.73	321.67	64.67	0	0	465.90	277.67	49.00	34.33	0	0	462.27	271.67	37.33	33.33	0
	0.7	1	882.13	350.00	70.67	3.70	1	700.15	313.33	57.00	31.33	2.50	1	795.21	314.00	47.67	32.67	1.32
	0.9	2	1530.16	365.78	73.33	7.73	1	1077.43	311.67	56.00	32.67	5.05	1	1002.20	306.00	44.33	35.11	3.52
	Ave/To	3	729.48	332.36	64.87	6.39	2	519.78	257.93	46.27	32.73	3.78	2	514.38	282.13	39.40	33.15	2.42

n	ϑ	NGR						ALL							
		No. Uns	Ave. Time (seconds)	Ave. Col	N0	N2	Ave. rGap	No. Uns	Ave. Time (seconds)	Ave. Col	N0	N1	N2	N3	Ave. rGap
35	0.1	0	59.94	260.30	11.67	35.50	0	0	40.20	163.33	11.67	32.33	60.00	59.33	0
	0.3	0	180.91	194.67	31.67	40.11	0	0	58.57	190.67	35.33	30.67	54.33	70.33	0
	0.5	0	311.34	224.33	32.33	45.67	0	0	107.83	227.33	12.67	32.44	22.00	77.00	0
	0.7	1	650.84	252.33	37.33	48.22	1.15	0	558.70	247.00	19.33	31.33	26.67	76.12	0
	0.9	1	805.96	275.56	40.44	55.78	2.68	1	600.61	265.33	20.67	30.67	28.56	74.11	1.83
	Ave/To	2	401.80	241.44	30.69	45.06	1.92	1	273.18	218.73	19.93	31.49	38.31	71.37	1.83

Table 4
Effectiveness of the SR inequalities.

n	ϑ	BP algorithm					BPC algorithm				
		No. Uns	Ave. Time (seconds)	Ave. Node	Ave. Gap	Ave. rCost	No. Uns	Ave. Time (seconds)	Ave. Node	Ave. Gap	Ave. rCost
35	0.1	0	198.85	286.11	0	251.62	0	205.15	62.33	0	260.13
	0.3	1	1344.97	665.11	0.01	250.10	0	1189.07	173.22	0	257.08
	0.5	1	1817.11	513.78	0.02	248.56	0	1532.70	108.33	0	253.66
	0.7	2	1562.00	135.67	0.20	246.61	1	1464.27	84.67	0.09	250.04
	0.9	2	2763.05	130.89	33.83	245.81	2	2693.74	30.33	3.30	248.05
	Ave/To	6	1537.20	346.31	6.81	248.54	3	1416.99	91.78	0.69	253.79

heuristic, only Tabu search, only dynamic *ng*-route relaxation, and all enhancement strategies, respectively.

Based on the above analysis, we included all the enhancements in the CG algorithm in the remaining experiment studies.

5.2.2. Effectiveness of the SR inequalities

We now test the effectiveness of the SR inequalities by comparing the developed BPC algorithm with the algorithm without the SR inequalities, denoted as BP algorithm. For each run of the comparison algorithms with 35 customers under different ϑ, we set a time limit of 7200 seconds.

Table 4 reports the "No. Uns", "Ave. Time", "Ave. Gap", the average number of nodes explored by the search tree prior to termination ("Ave. Node"), and the average objective value of the LMP at the root node ("Ave. rCost").

The results clearly demonstrate the effectiveness of the SR inequalities. Specifically, the use of the SR inequalities enables the optimal solving of 42 out of the 45 instances, whereas the algorithm without the SR inequalities can only solve 39 instances. In addition, the use of the SR inequalities significantly reduces the average optimality gap for the instances that cannot be optimally solved. For example, for $\vartheta = 0.9$, the hardest case to solve, the average optimality gap from the BPC algorithm is only 3.30%, whereas the average optimality gap from the BP algorithm reaches up to 33.83%. From column "Avg. rCost", we see that using the SR inequalities enhances the lower bound on the root node by 2.14%. It is well-known that with better lower bounds, the branch-and-bound algorithm is capable of pruning more nodes. This is clearly verified by the column "Ave. Node", from which we see that the average number of nodes from the BP algorithm is about four times as large as that from the BPC algorithm. With the aid of this improvement, significant computational time saving is attained using the SR inequalities. Indeed, approximately 7.82% of computational time saving on average is obtained from using the SR inequalities.

5.3. Performance of the BPC algorithm

In this section we assess the performance of the proposed BPC algorithm by comparing it directly with solving the AB-MILP using CPLEX and the BPC algorithm where each pricing problem is solved by CPLEX (denoted by BPC-CPLEX). We performed the experiments on all the instances. For the maximum time limit, we set 7200 seconds for instances with 10 to 35 customers, and 10,800 seconds for instances with 40 and 45 customers.

In Table 5 we report "No. Uns", "Ave. Time", "Ave. Gap", "Ave. Node", and the average computational times spent in solving the RLMPs at the root node ("AM. Time") and in solving the pricing problem at the root node ("AS. Time") for the BPC algorithm. The results clearly demonstrate that the BPC algorithm is far superior to CPLEX and the BPC-CPLEX algorithm. Indeed, CPLEX can only optimally solve 32 out of the 270 instances, and the BPC-CPLEX algorithm is able to optimally solve 35 out of the 270 instances within the time limit, which performs better than CPLEX. For instances with 20 customers, CPLEX and the BPC-CPLEX algorithm cannot even find a feasible solution due to out of memory. On the other hand, the BPC algorithm is capable of optimally solving 255 out of the 270 instances, and the average optimality gap for the unsolved instances falls below 4.54% in most cases and only reaches 7.53% (resp., 12.81%) for instances with 45 customers and $\vartheta = 0.7$ (resp., 0.9). In addition, for the instances optimally solved by all the three algorithms, the BPC algorithm shortens the computational time by at least three orders of magnitude on average, compared with CPLEX and the BPC-CPLEX algorithm. The results by comparing the BPC algorithm with the BPC-CPLEX algorithm also highlight the effectiveness of our solution strategy for solving the pricing problem.

As expected, we also see that the problem becomes dramatically difficult to solve with increasing number of customers and increasing ratio of customers that can be served by drones. Indeed, the average computational time significantly increases and more instances cannot be optimally solved within the time limit with increasing n and ϑ.

When comparing the relative time spent in solving the RLMPs versus solving the pricing sub-problems from the columns "AM. Time" and "AS. Time", it is evident that the bottleneck for large-sized instances is the pricing problem. The high computational time for solving the pricing problem is mainly because each pricing problem is the classical shortest path problem with time windows involving collaborative delivery by a pair of truck and drone, which is extremely challenging to solve. Therefore, finding approaches to efficiently solve the pricing problem can enhance the performance of the developed BPC algorithm.

Overall, these results clearly demonstrate the superiority of our BPC algorithm over the general-purpose solver CPLEX and the BPC-CPLEX algorithm, and the effectiveness of our BPC algorithm for solving the TD-RPTW, especially for large-sized instances.

5.4. Sensitivity analysis

In this section we analyze the impacts of the key model parameters on the performance of the BPC algorithm and the solution structure. To this end, we take the instances with 35 customers and $\vartheta = 0.5$ as examples to illustrate the changes in performance with varying parameters.

To compare the changes in the algorithm performance and solution structure, we consider the following four metrics: (i) the average total operating cost ("A. Cost"); (ii) the maximum number of customers served by drones ("M. DS"); (iii) the average computational time ("A. Time"); and (iv) the ratio of the number of instances solved ("O. Ratio") within a time limit of 10,800 seconds.

5.4.1. Impacts of the maximum flight time and carrying capacity of drones

We first discuss the impacts of the maximum flight time of the drone L^d and the maximum carrying capacity of the drone Q^d on the considered metrics. We depict the comparison results in Fig. 7,

Table 5
Performance of the BPC algorithm.

n	ϑ	BPC						CPLEX			BPC-CPLEX		
		No. Uns	Ave. Time (seconds)	Ave. Node	AM. Time	AS. Time	Ave. Gap	No. Uns	Ave. Time (seconds)	Ave. Gap	No. Uns	Ave. Time (seconds)	Ave. Gap
10	0.1	0	2.56	0.01	0.13	0.35	0	0	681.01	0	0	250.57	0
	0.3	0	2.78	0.01	0.29	0.57	0	0	898.53	0	0	649.93	0
	0.5	0	2.78	0.01	0.51	0.78	0	0	1408.98	0	0	837.82	0
	0.7	0	4.56	0.01	0.69	1.09	0	6	2085.25	3.80	4	1037.00	2.67
	0.9	0	4.56	0.01	1.01	1.49	0	7	2110.90	4.39	6	1104.28	3.86
20	0.1	0	15.89	0.02	0.69	3.22	0	/	/	/	9	7239.27	35.99
	0.3	0	7.22	0.02	2.95	6.62	0	/	/	/	/	/	/
	0.5	0	14.11	0.02	9.68	18.62	0	/	/	/	/	/	/
	0.7	0	11.44	0.02	13.85	24.05	0	/	/	/	/	/	/
	0.9	0	9.44	0.02	20.92	16.75	0	/	/	/	/	/	/
30	0.1	0	25.33	0.08	20.45	117.25	0	/	/	/	/	/	/
	0.3	0	30.89	0.08	40.92	336.78	0	/	/	/	/	/	/
	0.5	0	7.22	0.09	60.74	598.01	0	/	/	/	/	/	/
	0.7	0	2.33	0.11	500.74	1076.80	0	/	/	/	/	/	/
	0.9	1	2.25	0.11	300.52	969.85	3.00	/	/	/	/	/	/
35	0.1	0	62.33	0.11	39.08	205.15	0	/	/	/	/	/	/
	0.3	0	173.22	0.10	54.57	1189.07	0	/	/	/	/	/	/
	0.5	0	108.33	0.07	97.83	1532.70	0	/	/	/	/	/	/
	0.7	1	84.67	0.09	554.70	1464.27	0.09	/	/	/	/	/	/
	0.9	2	30.33	0.16	590.05	2693.73	3.30	/	/	/	/	/	/
40	0.1	0	71.67	0.07	68.00	253.05	0	/	/	/	/	/	/
	0.3	0	204.33	0.08	250.12	1323.57	0	/	/	/	/	/	/
	0.5	0	63.89	0.09	630.77	1573.99	0	/	/	/	/	/	/
	0.7	1	14.25	0.09	1314.30	1805.22	0.04	/	/	/	/	/	/
	0.9	2	107.75	0.10	1315.01	2728.29	4.54	/	/	/	/	/	/
45	0.1	0	38.33	0.11	757.94	1389.96	0	/	/	/	/	/	/
	0.3	1	129.88	0.12	1711.96	3584.92	1.50	/	/	/	/	/	/
	0.5	2	118.14	0.13	1978.45	5501.66	4.13	/	/	/	/	/	/
	0.7	2	60.14	0.13	3434.74	5395.29	7.53	/	/	/	/	/	/
	0.9	3	71.83	0.13	3467.77	7067.14	12.81	/	/	/	/	/	/

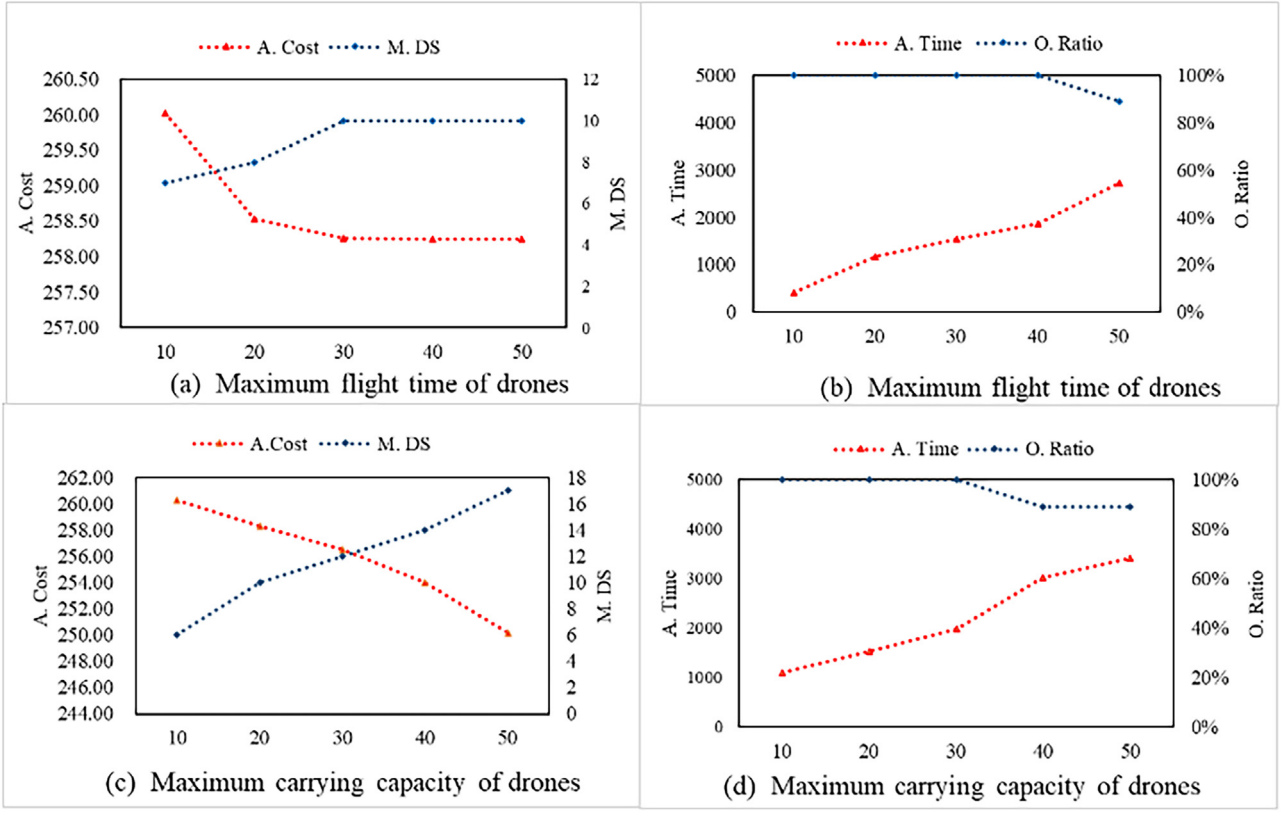

Fig. 7. Results on instances with different flight time and carrying capacity of the drone.

which demonstrates variations of "A. Cost", "M. DS", "A. Time", and "O. Ratio" with L^t, $Q^d \in \{10, 20, 30, 40, 50\}$, respectively.

We observe from Fig. 7(a) that, as L^d increases, "A. Cost" decreases and "M. DS" increases. Specifically, the rate of decrease in "A. Cost" tapers off and eventually evens out after a rapid decrease, while "M. DS" increases relatively steadily when $L^d \leq 30$ and remains constant when $L^d \geq 30$. Figure 7(b) indicates that "A. Time" increases linearly with L^d, increasing at least four times when L^d changes from 10 to 50. On the other hand, "O. Ratio" always equals 100% when $L^d \leq 40$ and drops to about 88% when $L^d = 50$, implying that we can optimally solve all the instances within the time limit when $L^d \leq 40$ but only 88% of the instances when $L^d = 50$. This is due to the fact that an increase in the maximum flight time limit will naturally increase the number of customers that can be visited per drone route, and thus enlarge the solution space. Therefore, with a larger L^d, we have a greater chance to reduce the total operating cost, but require more computational time to find the optimal solution. However, there exists a threshold of $L^d = 30$, beyond which "A. Cost" reduces very little and "M. DS" remains at about ten, implying that the drones can serve at most about 10 of the 18 customers in set D due to the time window constraint and the maximum carrying capacity limit of the drone.

Figure 7 (c) and (d) show the variations of the metrics with Q^d. From Fig. 7(c), we observe that "A. Cost" decreases and "M. DS" increases almost linearly with Q^d. Specifically, changing Q^d from 10 to 50 leads to a reduction in "A. Cost" by about \$8, which is significantly higher than that when changing L^d from 10 to 50. Moreover, all the 18 customers in set D can be served by drones with a maximum carrying capacity of 50. Figure 7(d) shows similar change patterns for "A. Time" and "O. Ratio" with varying L^d. The impact of increasing Q^d on "A. Time" is almost linear, where "A. Time" increases by about 2.5 times when Q^d changes from 10 to 50. Regarding "O. Ratio", all the instances can be optimally solved within the time limit when $L^d \leq 30$, and about 90% of the instances when $L^d = 40$ and 50. This is because the larger the carrying capacity of the drone is, the more are the customers whose demand can be satisfied per drone route, so the larger the solution space.

Overall, the above results demonstrate that the maximum flight time and carrying capacity of the drones have significant impacts on all the considered metrics. In addition, under the current parameter setting, increasing the maximum carrying capacity of the drones is more beneficial, which yields more operating cost saving at the expense of a relative little increase in the computational time.

5.4.2. Impacts of travel time ratio and time window width

We now discuss the impacts of the travel time ratio of drone over truck and the time window width on the considered metrics by varying β and δ in the ranges of $\{1.5, 2, 2.5\}$ and $\{-0.2, 0, 0.2\}$, respectively, where we set the time window width at $(1 + \delta)$ times of that of the baseline. Here, we assume that the speed of the drone remains the same. Thus, a higher β means a slower speed of the truck, probably due to road congestion.

We depict the comparison results in Fig. 8. From Fig. 8(a), we see that both "A. Cost" and "M. DS" increase with β, where "A. Cost" increases in an almost linear fashion and "M. DS" becomes stable when $\beta \geq 2$. This is because bad road conditions will increase the travel time of the truck, resulting in a slight increase in the number of customers that the drones serve and a dramatic increase in the total travel cost. Figure 8(b) indicates that the problem becomes easier to solve with increasing β. The reason is that the slower the truck, the later the truck arrives at the customer, the more likely it will violate the time window, resulting in fewer customers visiting. This will reduce the computational time to solve the pricing problem, so reducing the computational time of the algorithm.

Figure 8 (c) and (d) show how the metrics change with δ. The results demonstrate that "A. Cost" significantly decreases and "A. Time" sharply increases with increasing δ. Indeed, all the instances can be optimally solved when $\delta \leq 0$, while only about 80% of the instances can be optimally solved within the time limit when $\delta = 0.2$. The possible reason may be that a synthetic-route can serve more customers with a wider time window. This results in increasing the search space of the pricing problem, which will increase the computational time, and leads to decreases in the total travel time of the trucks and drones, and to decreases in the numbers of trucks and drones used, which will reduce the total operating cost.

5.4.3. Gain of the truck-based drone delivery over the truck-only delivery

In this section we analyze the gain of the truck-based drone delivery over the truck-only delivery, whose corresponding problem is the VRPTW. We use the instances with 35 customers as examples to illustrate how the average cost reduction in percentage of the truck-based drone delivery over the truck-only delivery changes when both Q^d and L^d vary in the range of $\{10, 30, 50\}$.

We depict the results in Fig. 9. It can be seen that significant cost saving is achieved from considering the truck-based drone delivery. Indeed, at least 4% of the cost saving from the truck-based drone delivery is obtained under the considered parameter setting. It is also interesting to see that the average cost reduction increases with Q^d and L^d, which is consistent with what we observed in Section 5.4.1. This is because, with larger Q^d and L^d, we may have more opportunities to make good choices with the help of the fast delivery advantage of the drones.

As expected, with a smaller L^d (resp., Q^d), the scope for cost reduction is limited when increasing Q^d (resp., L^d). This may be due to the maximum flight time (resp., carrying capacity) limit, even if the drone can satisfy the demands of more customers (resp., visit more customers) per drone route, the drone cannot serve all of them on one route. We further observe a common change pattern when increasing Q^d: the average cost saving remains stable beyond $Q^d = 30$. This observation indicates that using the drone with a maximum load of 30 is enough to reduce the total operating cost under the considered parameter setting.

6. Conclusions

The increasing requirements of logistics distribution highlight the need to deploy drones to deliver products to customers to enhance service quality and reduce delivery costs. We consider the truck-based drone delivery routing problem with time windows, where each truck is associated with a drone, and the drone can take off from its associated truck at a node, independently serves one or more customers within the customers' time windows, and then rejoins the truck at one node along the truck route. To solve the problem, we develop an enhanced BPC algorithm incorporating several enhancement strategies, where the SR inequalities are identified to tighten the lower bound, and a hybrid algorithm is devised to solve the pricing problem. The results of extensive numerical studies show that the enhancement strategies can dramatically improve the performance of the algorithm and the developed algorithm significantly outperforms CPLEX and the BPC-CPLEX algorithm. The results comparing the truck-based drone delivery with the truck-only delivery demonstrate the gain of the former means of delivery. We also perform sensitivity analysis of the impacts of the key model parameters on the performance of the proposed BPC algorithm and the solution structure, generating practical insights from the research findings.

For future research, it is noted that wind direction and speed can affect the travel time and battery usage of drones, and traf-

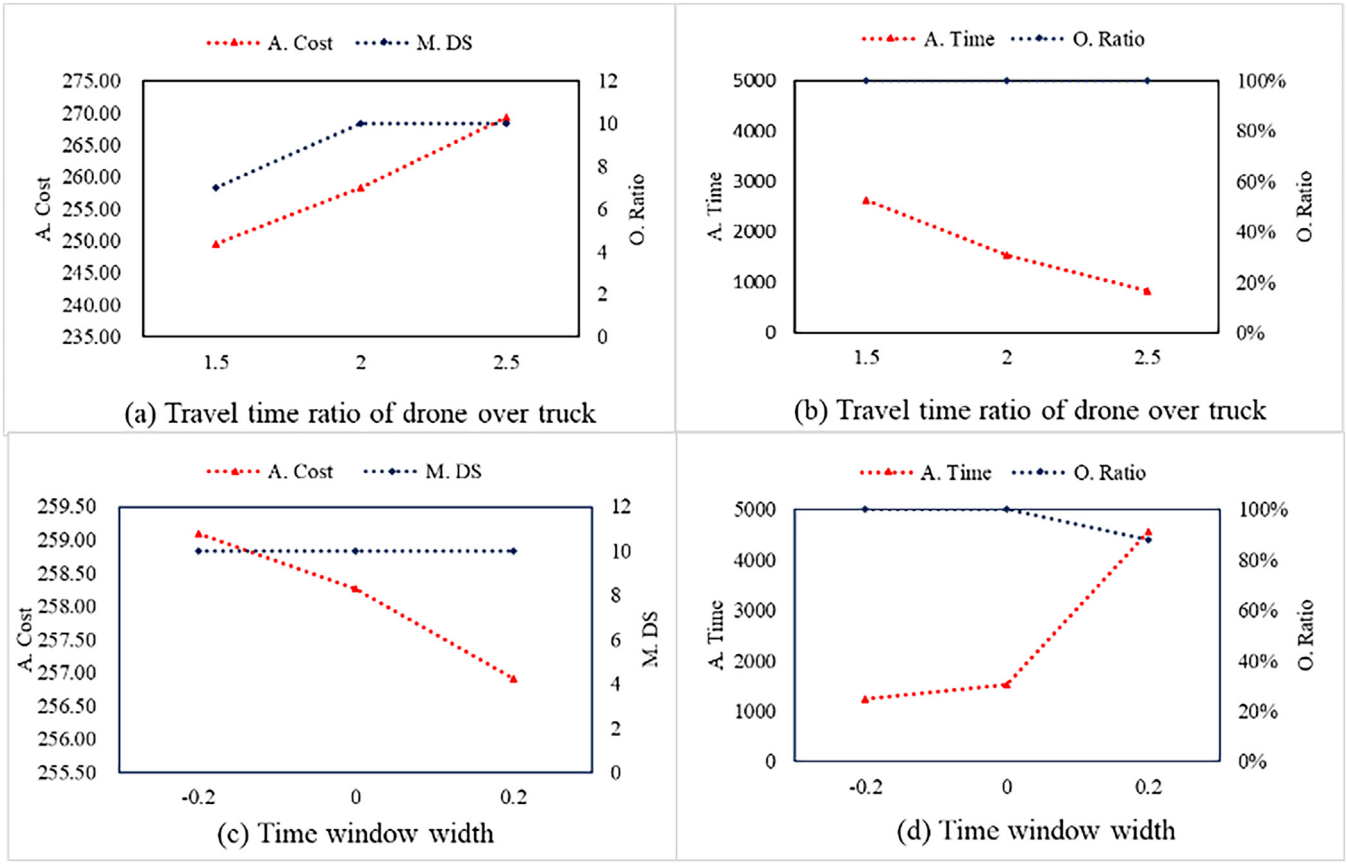

Fig. 8. Results on instances with different travel time rate and time window width.

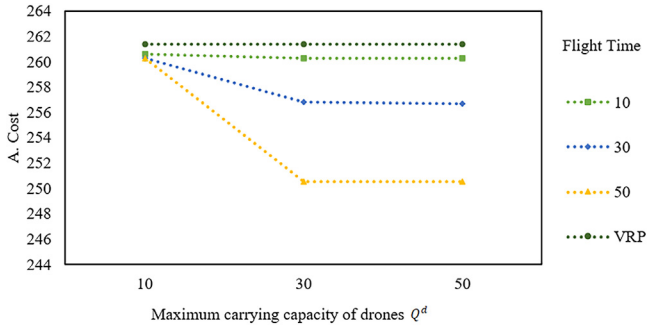

Fig. 9. Operating cost comparison of the truck-based drone delivery over the truck-only delivery.

fic congestion will affect the travel time of trucks. So it is worth studying the TD-DRPTW in an uncertain environment, which is closer to real practice. Moreover, the travel time of a drone heavily depends on its payload. Thus, it is also of interest to extend our model to consider the correlation between the travel time and payload of a drone.

Acknowledgements

This paper was supported in part by the National Natural Science Foundation of China under grant numbers 71971041, 72171161, and 71871148; by the Major Program of National Social Science Foundation of China under Grant 20&ZD084; by the special key project of Chengdu Philosophy and social science planning "Young Eagle Plan" for "Study and Interpret the Party's Twenty Great Spirits" (2022E04); by the Key Research and Development Project of Sichuan Province (No. 2023YFS0397); by the Chengdu Philosophy and Social Science Planning Project (No. 2022C19); and by the Sichuan University to Building a World-class University (No. SKSYL2021-08).

References

Agatz, N., Bouman, P., & Schmidt, M. (2018). Optimization approaches for the traveling salesman problem with drone. *Transportation Science, 52*(4), 965–981.

Baldacci, R., Mingozzi, A., & Roberti, R. (2011). New route relaxation and pricing strategies for the vehicle routing problem. *Operations Research, 59*(5), 1269–1283.

Bishop, T. (2015). Alibaba tests drone package delivery in China in three-day trial. https://www.geekwire.com/2015/alibaba-tests-drone-package-delivery-china-three-day-trial/. (Accessed July 21, 2021).

Boysen, N., Briskorn, D., Fedtke, S., & Schwerdfeger, S. (2018). Drone delivery from trucks: Drone scheduling for given truck routes. *Networks, 72*(4), 506–527.

Dayarian, I., Crainic, T. G., Gendreau, M., & Rei, W. (2015). A branch-and-price approach for a multi-period vehicle routing problem. *Computers and Operations Research, 55*, 167–184.

Dayarian, I., Savelsbergh, M., & Clarke, J. P. (2020). Same-day delivery with drone resupply. *Transportation Science, 54*(1), 229–249.

Desaulniers, G., Lessard, F., & Hadjar, A. (2008). Tabu search, partial elementarity, and generalized k-path inequalities for the vehicle routing problem with time windows. *Transportation Science, 42*(3), 387–404.

Di Puglia Pugliese, L., & Guerriero, F. (2017). Last-mile deliveries by using drones and classical vehicles. In *In optimization and decision science: Methodologies and applications: ODS, Sorrento, Italy, September 4–7, 2017 47* (pp. 557–565). Springer International Publishing.

Gaba, F., & Winkenbach, M. (2020). A systems-level technology policy analysis of the truck-and-drone cooperative delivery vehicle system. https://hdl.handle.net/1721.1/125416. (Accessed October 20, 2021).

Gendreau, M., Hertz, A., & Laporte, G. (1994). A tabu search heuristic for the vehicle routing problem. *Management Science, 40*(10), 1276–1290.

Gilchrist, K. (2017). World's first drone delivery service launches in Iceland. https://www.cnbc.com/2017/08/22/worlds-first-drone-delivery-service-launches-in-iceland.html. (Accessed August 17, 2021).

Global petrol prices (2018). https://www.globalpetrolprices.com/gasoline_prices/. (Accessed Aug. 5th, 2018).

Ha, Q. M., Deville, Y., Pham, Q. D., & Hà, M. H. (2018). On the min-cost traveling salesman problem with drone. *Transportation Research Part C: Emerging Technologies, 86*, 597–621.

Hawkins, A. J. (2022). Alphabet's Wing kicks off drone delivery service in Dallas on April 7th. https://www.theverge.com/2022/4/4/23006894/alphabet-wing-drone-delivery-dallas-kick-off. (Accessed July 21, 2022).

Jeong, H. Y., Song, B. D., & Lee, S. (2019). Truck-drone hybrid delivery routing: Payload-energy dependency and no-fly zones. *International Journal of Production Economics, 214*, 220–233. (Accessed July 21, 2022)

Jepsen, M., Petersen, B., Spoorendonk, S., & Pisinger, D. (2008). Subset-row inequalities applied to the vehicle-routing problem with time windows. *Operations Research, 56*(2), 497–511.

Kang, M., & Lee, C. (2021). An exact algorithm for heterogeneous drone-truck routing problem. *Transportation Science, 55*(5), 1088–1112.

Karak, A., & Abdelghany, K. (2019). The hybrid vehicle-drone routing problem for pick-up and delivery services. *Transportation Research Part C: Emerging Technologies, 102*, 427–449.

Kim, E. (2016). The most staggering part about Amazon's upcoming drone delivery service. https://www.businessinsider.com/cost-savings-from-amazon-drone-deliveries-2016-6. (Accessed July 21, 2018).

Kim, S., & Moon, I. (2019). Traveling salesman problem with a drone station. *IEEE Transactions on Systems, Man, and Cybernetics: Systems, 49*(1), 42–52.

Kitjacharoenchai, P., Min, B. C., & Lee, S. (2020). Two echelon vehicle routing problem with drones in last mile delivery. *International Journal of Production Economics, 225*, 107598.

Kitjacharoenchai, P., Ventresca, M., Moshref-Javadi, M., Lee, S., Tanchoco, J. M., & Brunese, P. A. (2019). Multiple traveling salesman problem with drones: Mathematical model and heuristic approach. *Computers And Industrial Engineering, 129*, 14–30.

Kloster, K., Moeini, M., Vigo, D., & Wendt, O. (2023). The multiple traveling salesman problem in presence of drone-and robot-supported packet stations. *European Journal of Operational Research, 305*(2), 630–643.

Kuo, R. J., Lu, S. H., Lai, P. Y., & Mara, S. T. W. (2022). Vehicle routing problem with drones considering time windows. *Expert Systems with Applications, 191*, 116264.

Landi, H. (2022). Walgreens, Alphabet's Wing kick off drone delivery service in major metro area. https://www.fiercehealthcare.com/health-tech/walgreens-alphabets-wing-kick-drone-delivery-service-major-metro-area. (Accessed July 21, 2022).

Luo, Z., Poon, M., Zhang, Z., Liu, Z., & Lim, A. (2021). The multi-visit traveling salesman problem with multi-drones. *Transportation Research Part C: Emerging Technologies, 128*, 103172.

Macrina, G., Pugliese, L. D. P., Guerriero, F., & Laporte, G. (2020). Drone-aided routing: A literature review. *Transportation Research Part C: Emerging Technologies, 120*, 102762.

McNabb, M. (2021). Matternet station makes drone delivery scalable, at a reasonable cost. https://dronelife.com/2021/10/01/matternet-station-makes-drone-delivery-scalable-at-a-reasonable-cost/. (Accessed July 21, 2022).

Minemyer, P. (2019). CVS, UPS join forces on drone delivery. https://www.fiercehealthcare.com/finance/cvs-ups-join-forces-drone-delivery. (Accessed March 10, 2021).

Murray, C. C., & Chu, A. G. (2015). The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C: Emerging Technologies, 54*, 86–109.

Pierce, D. (2013). Delivery drones are coming: Jeff Bezos promises half-hour shipping with Amazon Prime Air. https://www.theverge.com/2013/12/1/5164340/delivery-drones-are-coming-jeff-bezos-previews-half-hour-shipping. (Accessed July 21, 2022).

Righini, G., & Salani, M. (2006). Symmetry helps: Bounded bi-directional dynamic programming for the elementary shortest path problem with resource constraints. *Discrete Optimization, 3*(3), 255–273.

Roberti, R., & Mingozzi, A. (2014). Dynamic *ng*-path relaxation for the delivery man problem. *Transportation Science, 48*(3), 413–424.

Roberti, R., & Ruthmair, M. (2021). Exact methods for the traveling salesman problem with drone. *Transportation Science, 55*(2), 315–335.

Sacramento, D., Pisinger, D., & Ropke, S. (2019). An adaptive large neighborhood search metaheuristic for the vehicle routing problem with drones. *Transportation Research Part C: Emerging Technologies, 102*, 289–315.

Sanchez, A. (2017). UPS demonstrates truck-based drone system for rural deliveries. https://www.motortrend.com/news/1702-ups-demonstrates-truck-based-drone-system-for-rural-deliveries/. (Accessed July 21, 2022).

Solomon, M. M. (1987). Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research, 35*(2), 254–265.

Tamke, F., & Buscher, U. (2021). A branch-and-cut algorithm for the vehicle routing problem with drones. *Transportation Research Part B: Methodological, 144*, 174–203.

Thiago, T. (2019). Comparing the cost-effectiveness of drones v ground vehicles for medical, food and parcel deliveries. https://www.unmannedairspace.info/commentary/comparing-the-cost-effectiveness-of-drones-v-ground-vehicles-for-medical-food-and-parcel-deliveries. (Accessed July 21, 2022)

Wang, K., Yuan, B., Zhao, M., & Lu, Y. (2020). Cooperative route planning for the drone and truck in delivery services: A bi-objective optimisation approach. *Journal of the Operational Research Society, 71*(10), 1657–1674.

Wang, Z., & Sheu, J. B. (2019). Vehicle routing problem with drones. *Transportation Research Part B: Methodological, 122*, 350–364.

Wohlsen, M. (2014). The next big thing you missed: Amazon's delivery drones could work–they just need trucks. https://www.wired.com/2014/06/the-next-big-thing-you-missed-delivery-drones-launched-from-trucks-are-the-future-of-shipping/. (Accessed July 14, 2021).

Yurek, E. E., & Ozmutlu, H. C. (2018). A decomposition-based iterative optimization algorithm for traveling salesman problem with drone. *Transportation Research Part C: Emerging Technologies, 91*, 249–262.